Chap 5B

Trees

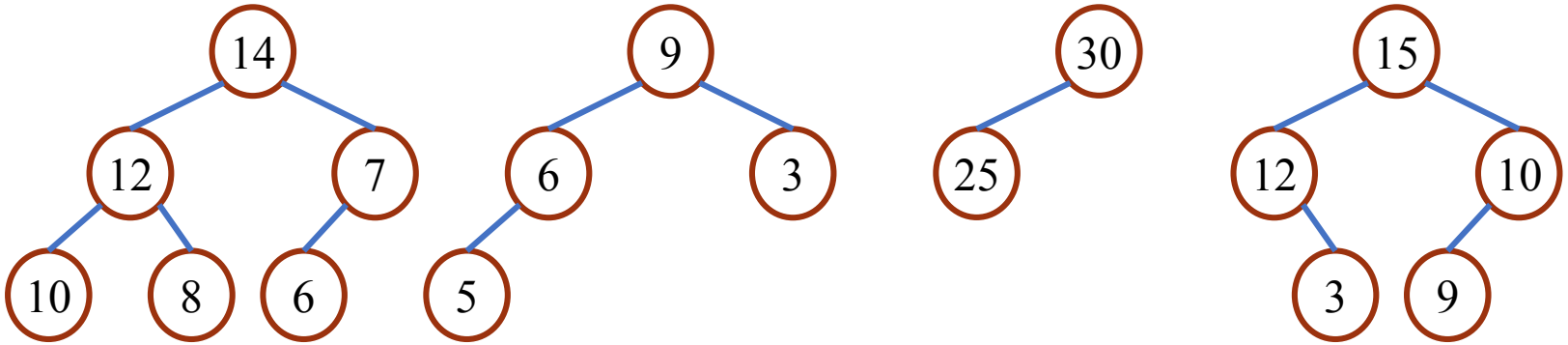
海大資訊工程系
蔡東佐

Outline

- Heaps（堆積）
- Binary Search Trees（二元搜尋樹）
- Selection Trees（選取樹）
- Forests（樹林）
- Representation of Disjoint Sets（集合的表示法）
- Counting Binary Trees（二元樹計數）

Definition of a Max Tree

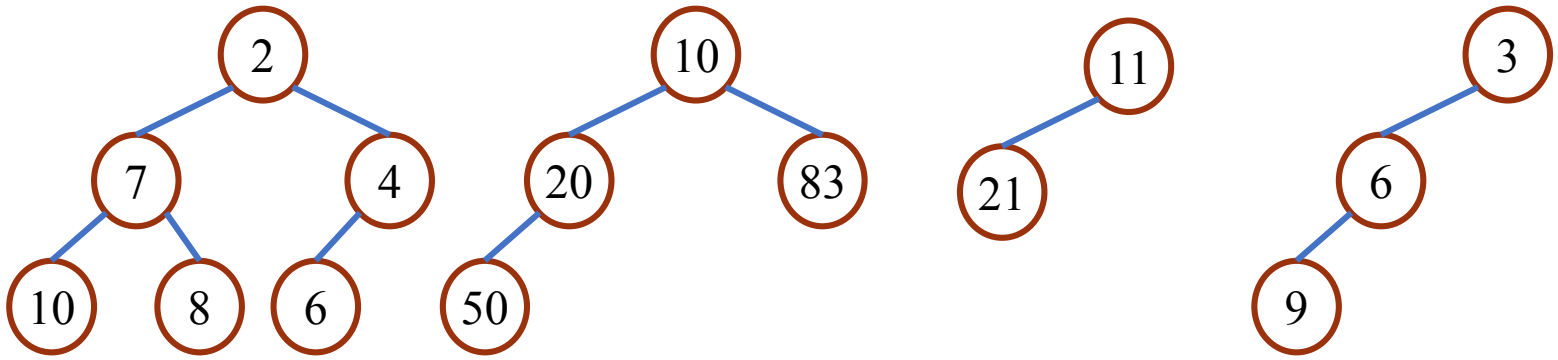
- A **max tree** is a tree in which
  - ✓the **key value** in each node is **no smaller** than the key values in its children if any.



Max Trees

Definition of a Min Tree

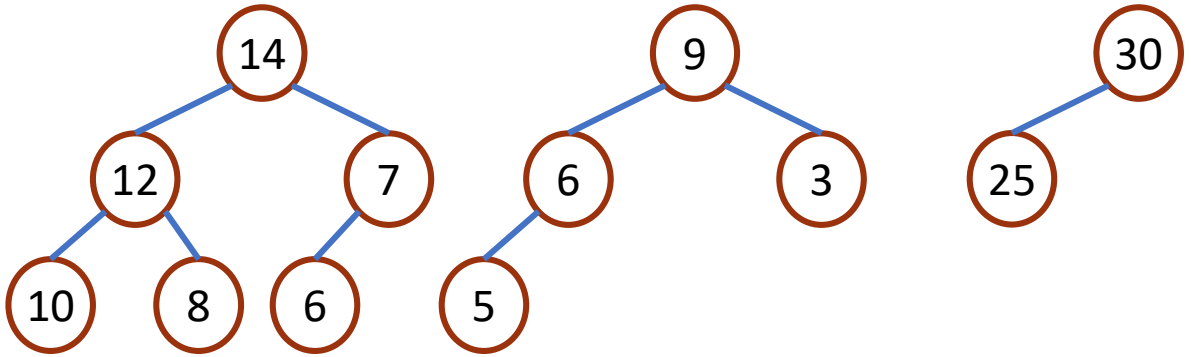
- A **min tree** is a tree in which
  - ✓the **key value** in each node is **no larger** than the key values in its children if any.



Min Trees

Definition of a Max Heap

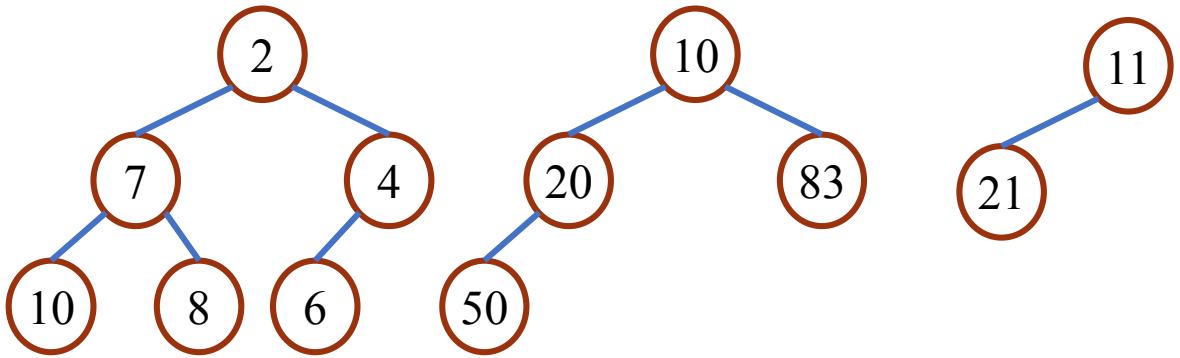
- A **max heap** is a **complete** binary tree that is also a max tree.



Max Heaps

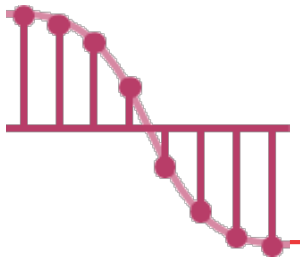
Definition of a Min Heap

- A **min heap** is a **complete** binary tree that is also a min tree.



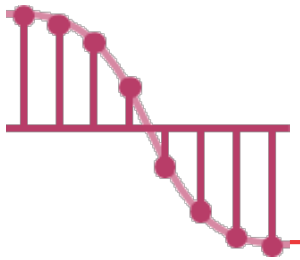
Min Heaps

完全二元樹(C.P.T):
只有最下面2層結点的度數
可以小於2,且最下面一層的結点
都集中在該層左边的連續位置上



Heaps and Priority Queues

- Heaps are frequently used to implement **priority queues**.
- In this kind of queue,
  - ✓the element to be **deleted** is the one with highest (or lowest) priority.
  - ✓at any time, an element with arbitrary priority can be **inserted** into the queue.

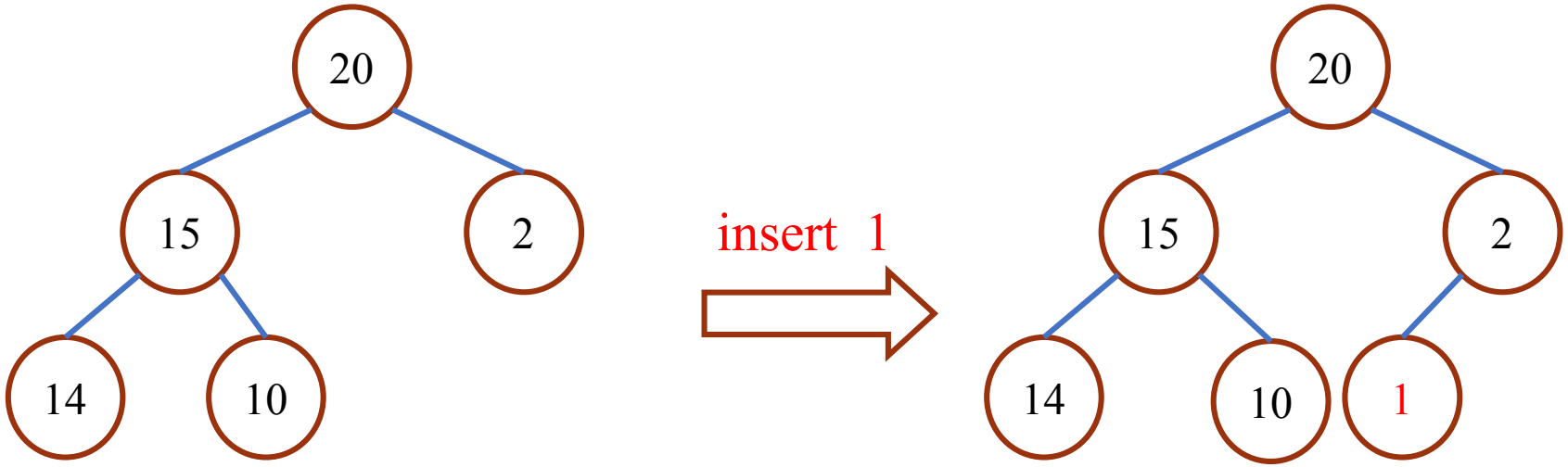


Why Priority Queue ?

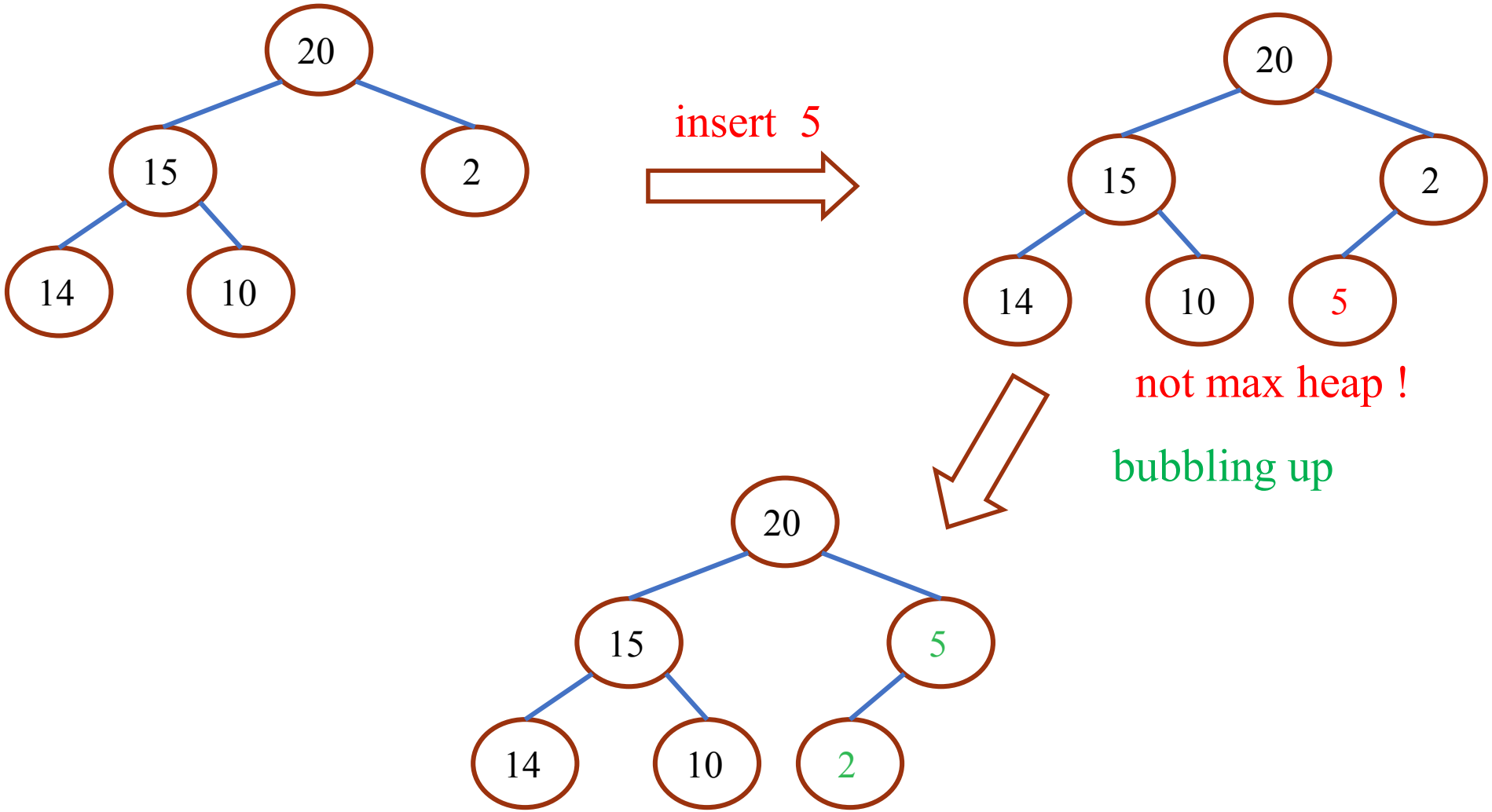
- Example 5.1: There is one machine that can be used by all customers.
  - ✓Case 1: Each customer pays the same money but needs different time to use it.
    - Use **min priority queue** to maximize the benefit.
  - ✓Case 2: Each customer needs the same time but pays different amount of money for using it.
    - Use **max priority queue** to maximize the benefit.

Insertion into a Max Heap

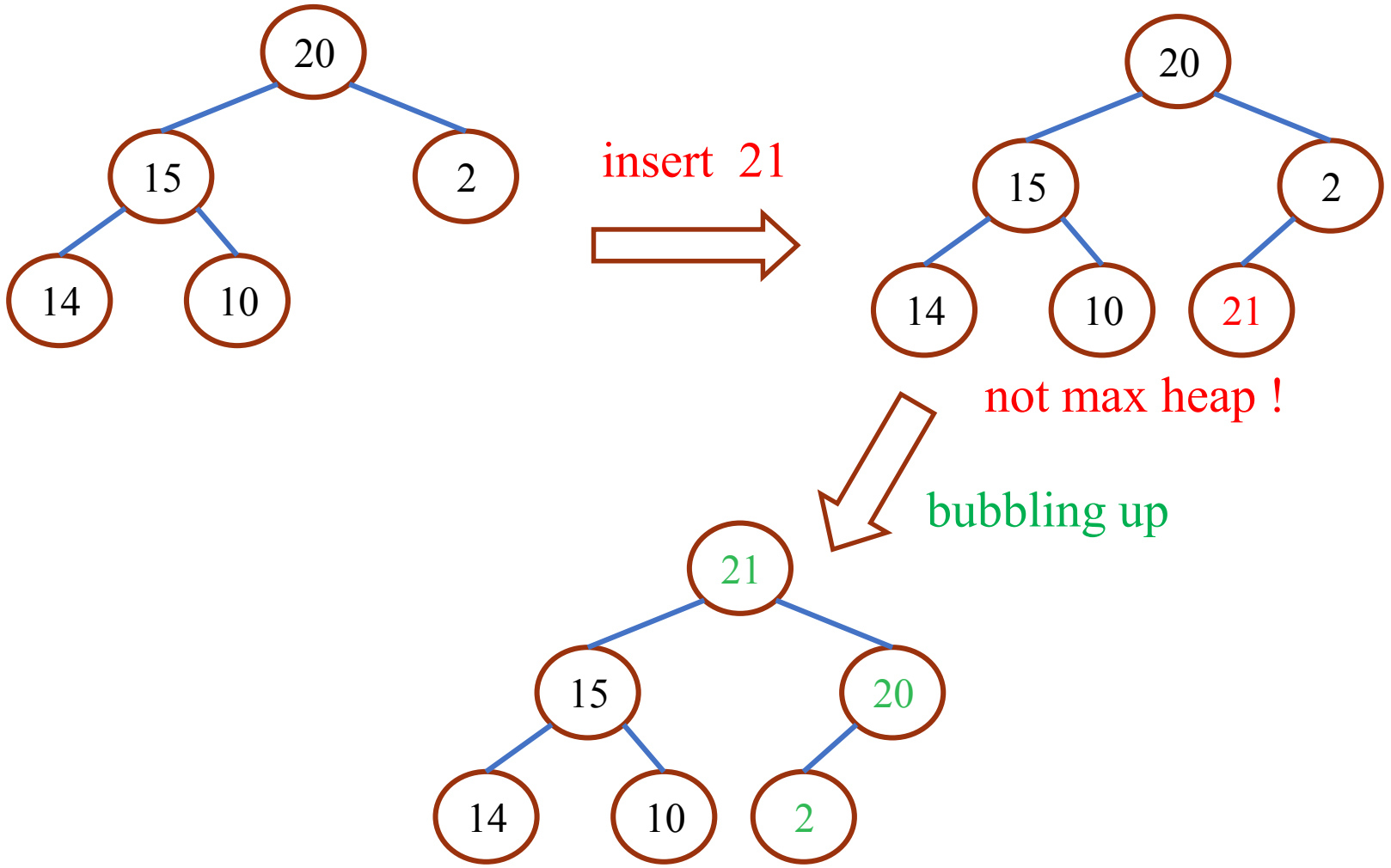
- In order to determine the correct place for the element that is being inserted, we use a **bubbling up** process.
- A **bubbling up** process begins at the new node of the tree and moves toward the root.



Insertion into a Max Heap – example 1



Insertion into a Max Heap – example 2



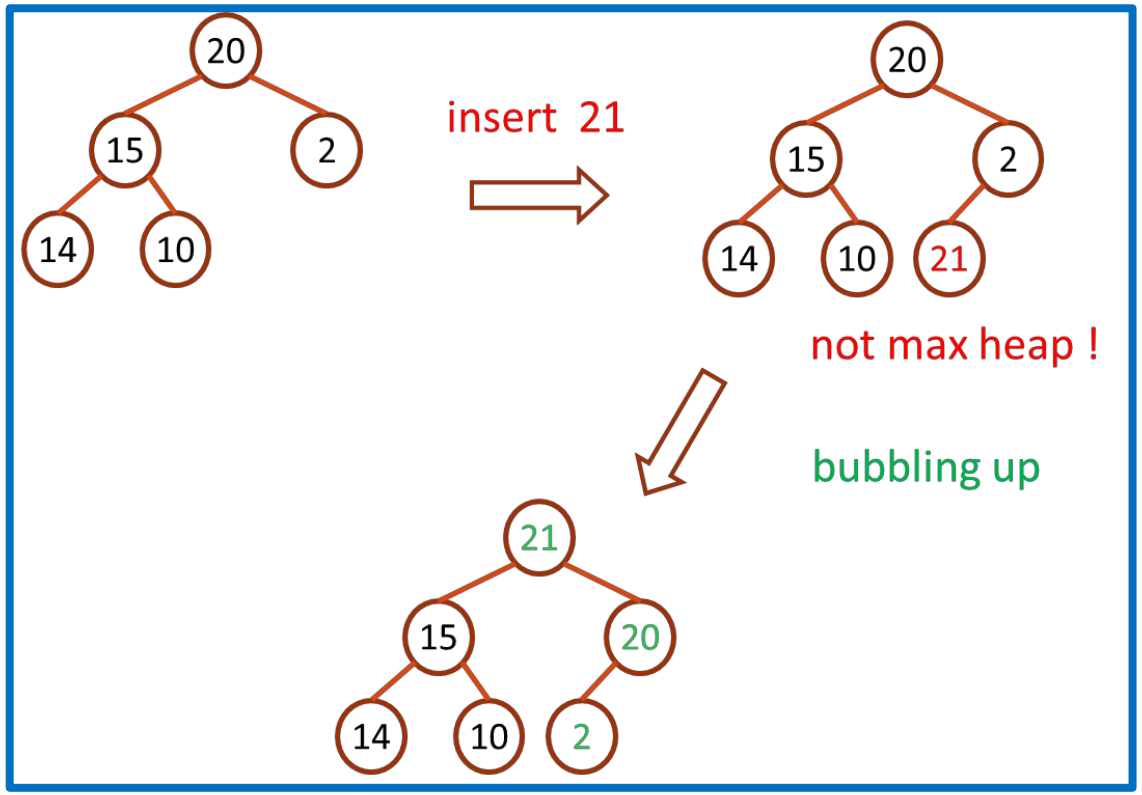
Program 5.13: Insert into a max heap 1/2

- We assume that the heap is created using the following C declarations:

```
#define MAX_ELEMENTS 200  /* maximum heap size+1 */
#define HEAP_FULL (n) (n == MAX_ELEMENTS −1)
#define HEAP_EMPTY (n)  (!n)
typedef struct {
        int key;
        /* other fields */
} element;
element heap[MAX_ELEMENTS];
int n = 0;
```

Program 5.13: Insert into a max heap 2/2

```
void push (element item, int *n)
{
   /* insert item into a max heap of current size *n */
   int i;
   if (HEAP_FULL (*n)) {                                   O(1)
         printf("The heap is full. \n");
         exit (EXIT_FAILURE);
   }
   i = ++ (*n);
   while ((i != 1) && (item.key > heap[i/2].key)) {        O(log2n)
         heap[i] = heap [i/2];
         i /= 2;
   }
   heap[i] = item;                                         O(1)
}
```

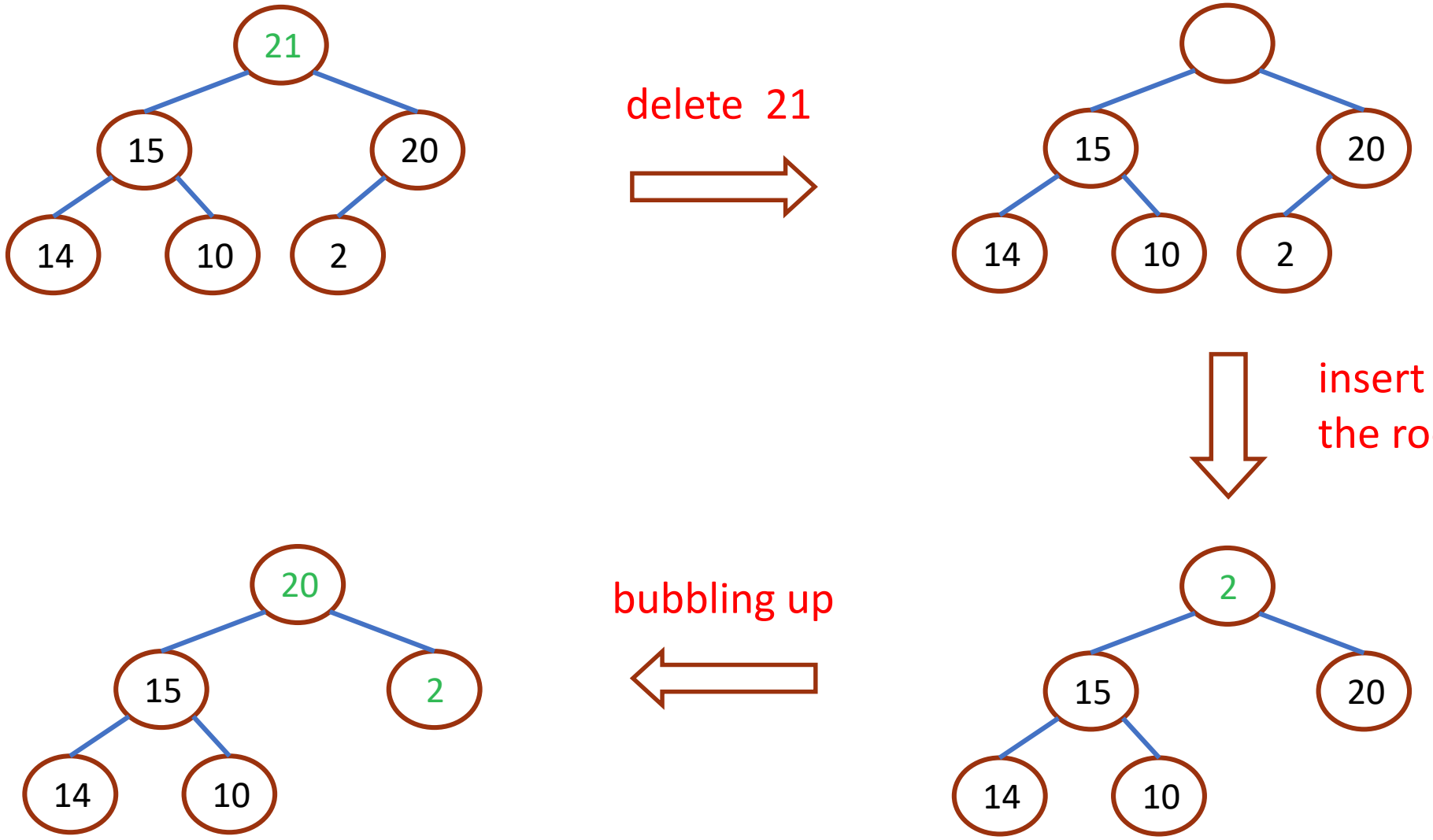


- The time complexity of the insertion function is $O(\log_2 n)$.

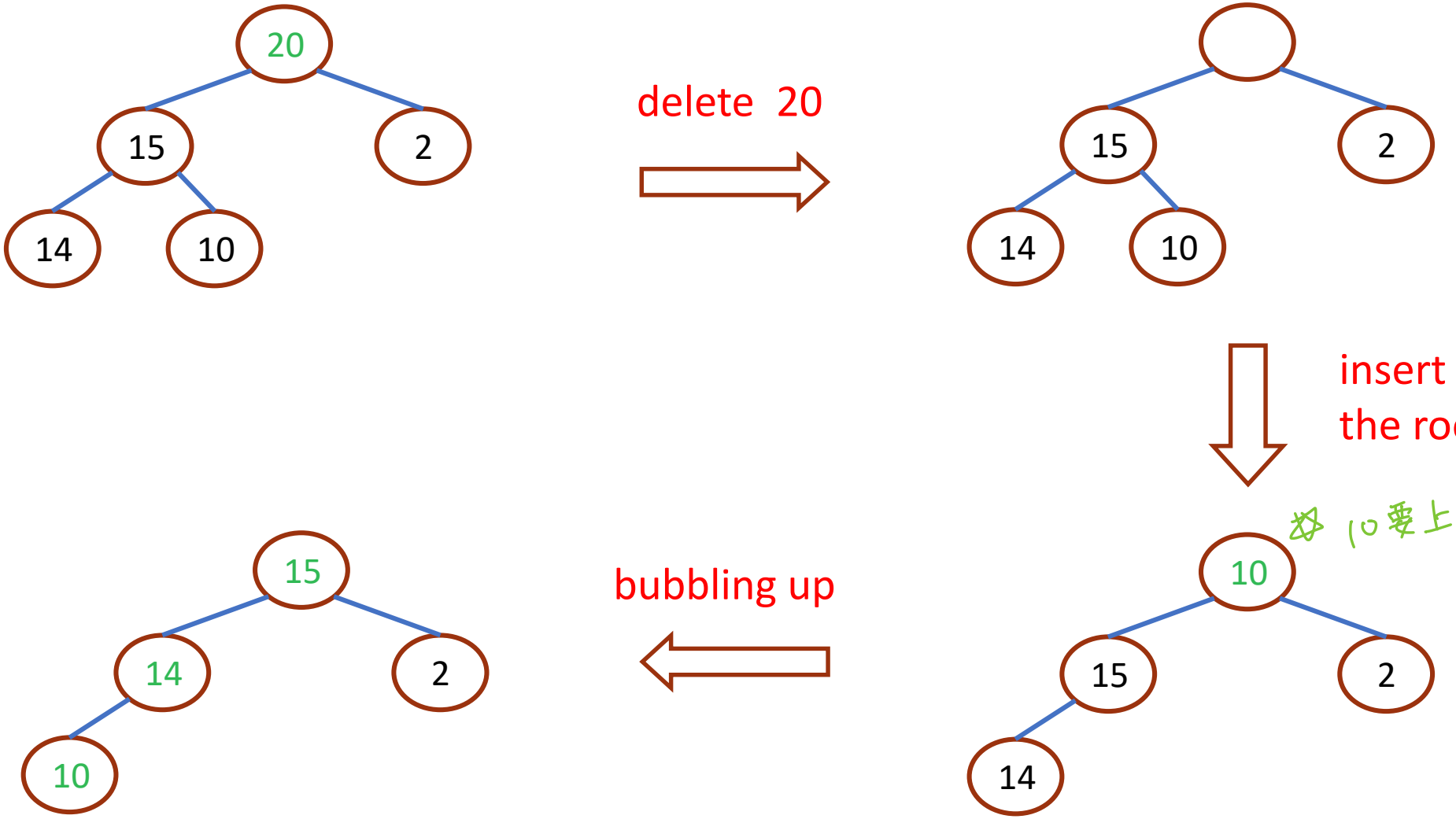
Deletion from a Max Heap 1/4

- When an element is to be deleted from a max heap, it is **always** taken from the root of the heap.
- The steps of deletion from a Max heap:
  - ✓delete the root node
  - ✓insert the **last** node into the root
  - ✓use the bubbling up process to ensure that the resulting heap remains a max heap

Deletion from a Max Heap 2/4

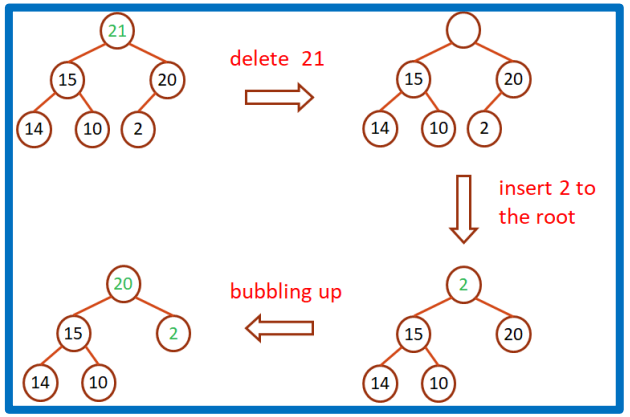


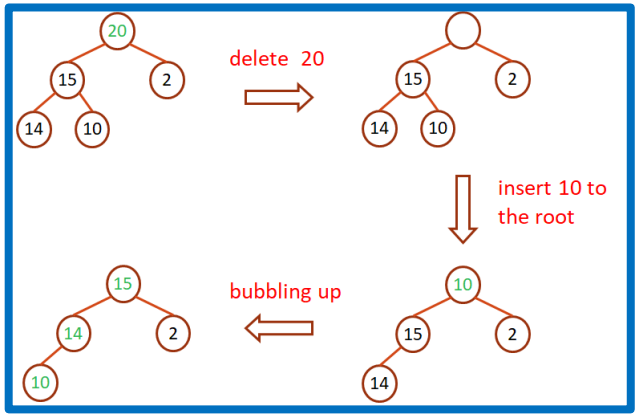
Deletion from a Max Heap 3/4

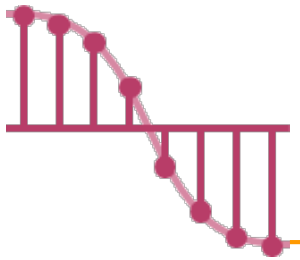


Deletion from a Max Heap 4/4

```
element pop(int *n)
{  /* delete element with the highest key from the
heap */
   int parent, child;
   element item, temp;
   if (HEAP_EMPTY(*n)) {
       fprintf(stderr, "The heap is empty\n");
       exit(EXIT_FAILURE);
   }
   /* save value of the element with the highest key */
   item = heap[1];
   /* use last element in heap to adjust heap */
   temp = heap[(*n)--];
   parent = 1;                                   O(1)
   child = 2;
   while (child <= *n) {                         O(log2n)
      /* find the larger child of the current parent */
      if ((child < *n) && (heap[child].key < heap[child+1].key))
          child++;
      if (temp.key >= heap[child].key)   break;
      /* move to the next lower level */
      heap[parent] = heap[child];
      parent = child;
      child *= 2;
   }
   heap[parent] = temp;                          O(1)
   return item;
}
```





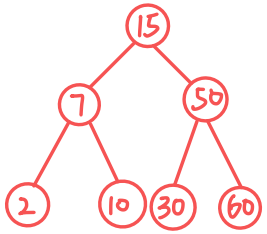


Binary Search Tree

- Binary search tree provides a **better performance** then any of the data structures studied so far when the functions to be performed are search, insertion and deletion.

Binary Search Tree

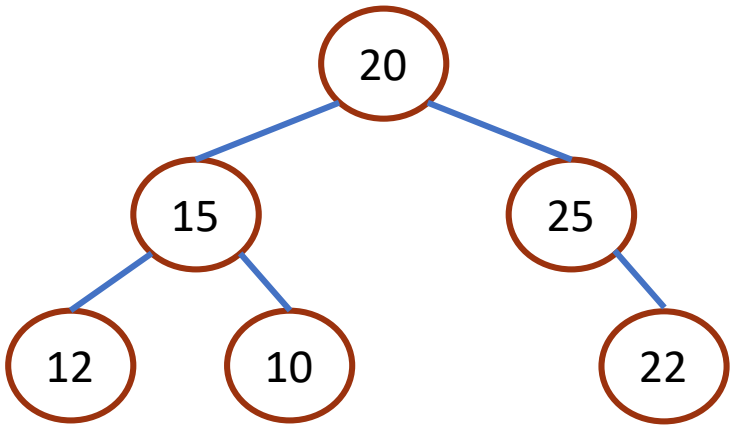
- A **binary search tree** is a binary tree. It may be empty. If it is not empty, then it satisfies the following properties:
  - ✓Each node has **exactly one key** and the keys in the tree are **distinct.**
  - ✓The keys (if any) in the **left** subtree are **smaller** than the key in the root.
  - ✓The keys (if any) in the **right** subtree are **larger** than the key in the root.
  - ✓The left and right subtrees are also binary search tree.



if have a number list:
2 7 10 15 30 50 60
construct a b tree

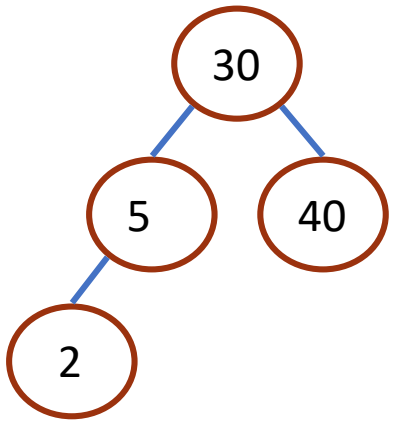
Figure 5.29 Binary trees

- Figure (a) is not a binary search tree.
- Figures (b) and (c) are binary search trees.



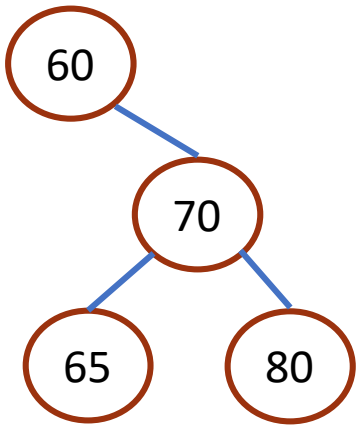
(a)

not a binary search tree



(b)

a binary search tree

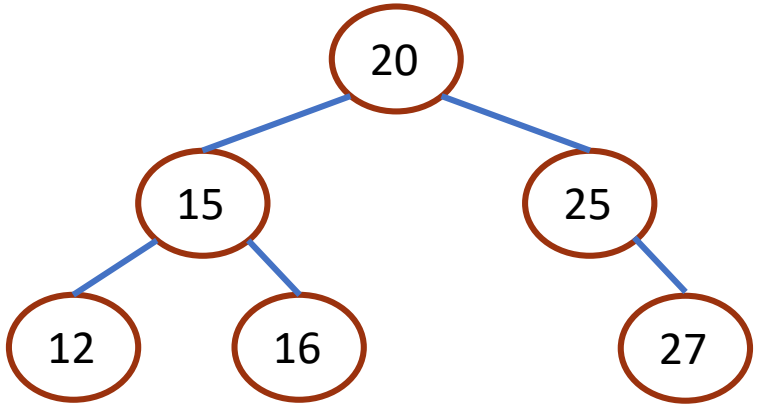


(c)

a binary search tree

Program 5.15
Recursive search of a binary search tree

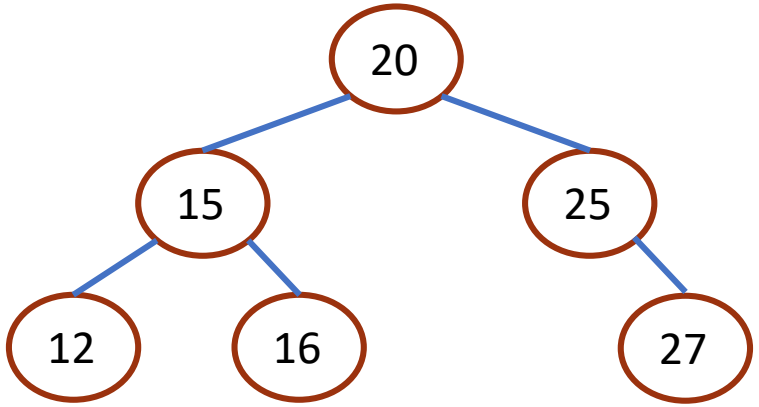
```
element* search(treePointer root, int key)
{
    /* return a pointer to the node that contains key,
    if there is no such node, return NULL. */
    if (!root) return NULL;
    if (k == root→data.key) return &(root→data);
    if (k < root→data.key)
        return search(root→leftChild, k);
    return search(root→rightChild, k);
}
```



- The time complexity of the search function is $O(h)$, where h is the height of the binary search tree.

Program 5.16
Iterative search of a binary search tree

```
element* iterSearch(treePointer tree, int k)
{
    /* return a pointer to the node that contains key,
    if there is no such node, return NULL. */
    while (tree) {
        if (k == tree→data.key) return &(tree→data);
        if (k < tree→data.key)
            tree = tree→leftChild;
        else
            tree = tree→rightChild;
    }
    return NULL;
}
```

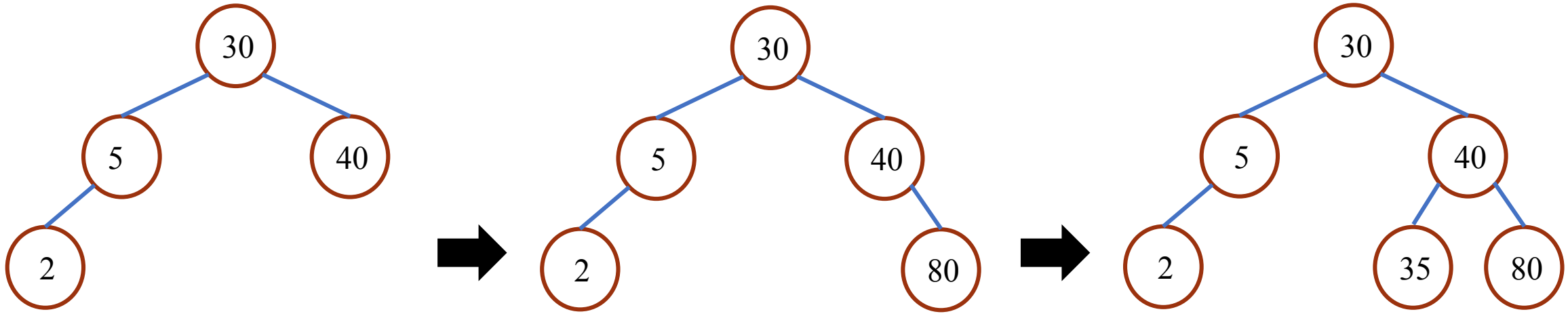


- The time complexity of the search function is $O(h)$, where h is the height of the binary search tree.

Figure 5.30
Inserting into a binary search tree

imagine a b.s.t is a number list: [2, 5, 30, 40]
35 80

insert 35, 80:



The original tree

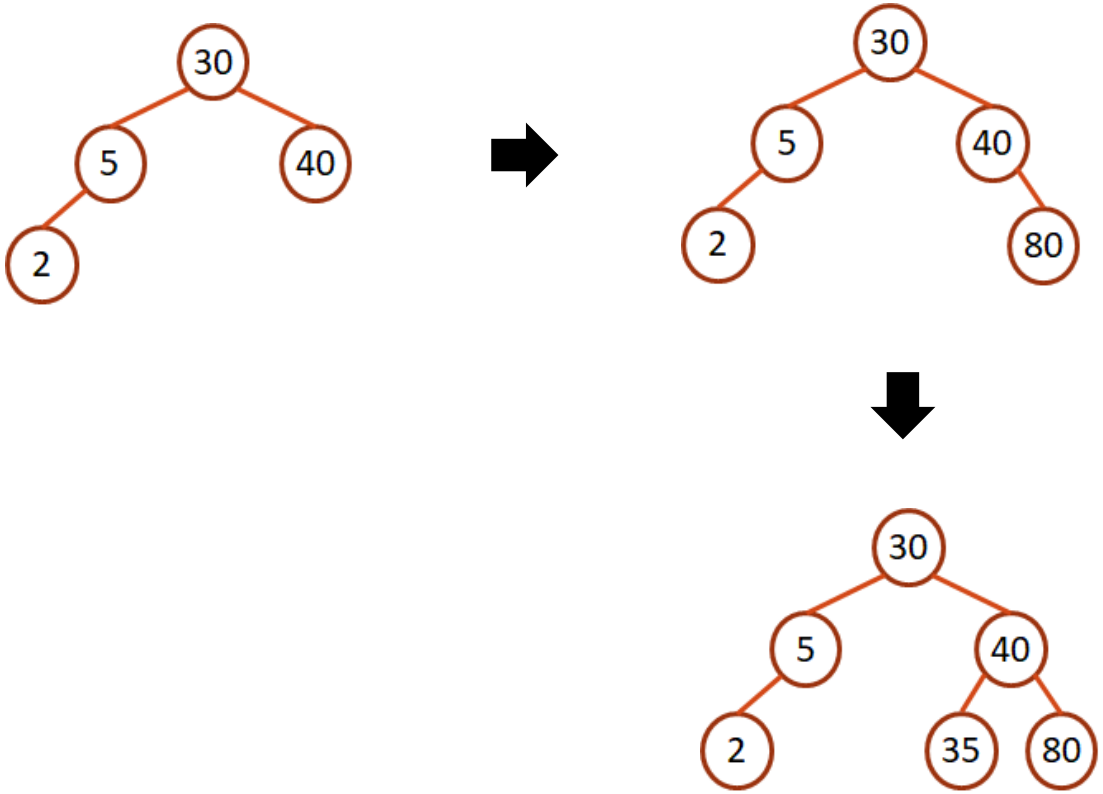
Insert 80

insert 35

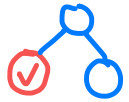
Program 5.17
Inserting a dictionary pair into a binary search tree

```
void insert(treePointer *node, int k, iType theItem)
{
    /* If k is in the tree pointed at by node do nothing;
    otherwise add a new node with data = (k, theItem) */
    treePointer ptr, temp = modifiedSearch(*node, k);
    if (temp || !(*node)) {    /* k is not in the tree */
        MALLOC(ptr, sizeof(*ptr));
        ptr->data.key = k;
        ptr->data.item = theItem;
        ptr->leftChild = ptr->rightChild = NULL;
        if (*node) /* insert as child of temp */
            if(k < temp->data.key)
                temp->leftChild = ptr;
            else
                temp->rightChild = ptr;
        else
            *node = ptr;
    }
}
```

modifiedSearch: 如果樹是空的或k存在回傳NULL; 否則回傳碰到的最後一個節點的指標。



Deletion from a Binary Search Tree

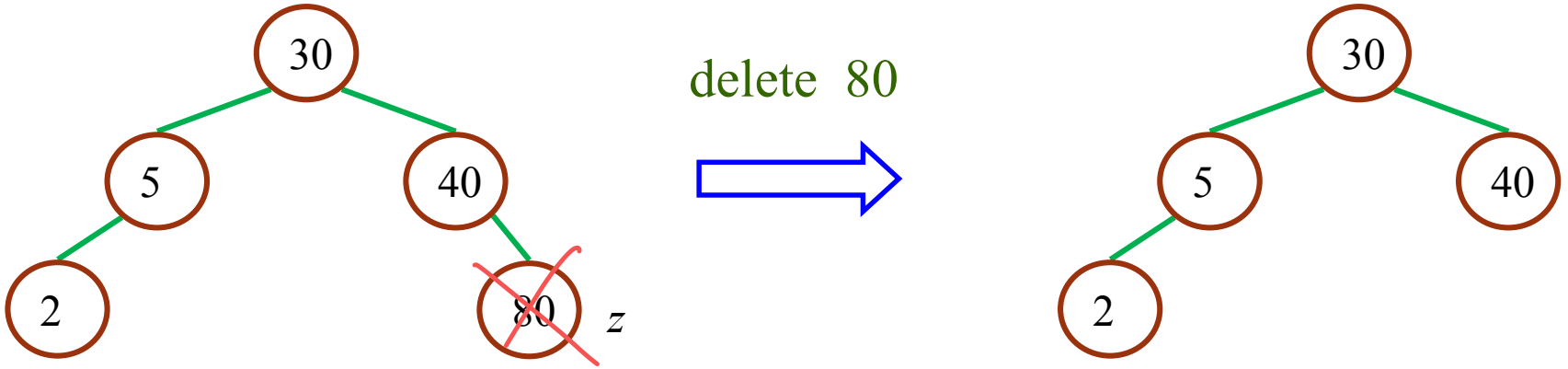
- Three cases should be considered
  - ✓**Case 1: leaf** 是葉子 
    - delete the node and set the pointer from the parent node to null
  - ✓**Case 2: has only one child** 
    - delete the node and change the pointer from the parent node to the single-child node
  - ✓**Case 3: has two children** 
    - replaced by the largest element in its left subtree, or
    - replaced by the smallest element in its right subtree

Three cases should be considered 1/3

➢Case 1: leaf

✓delete the node and set the pointer from the parent node to null

Case 1:



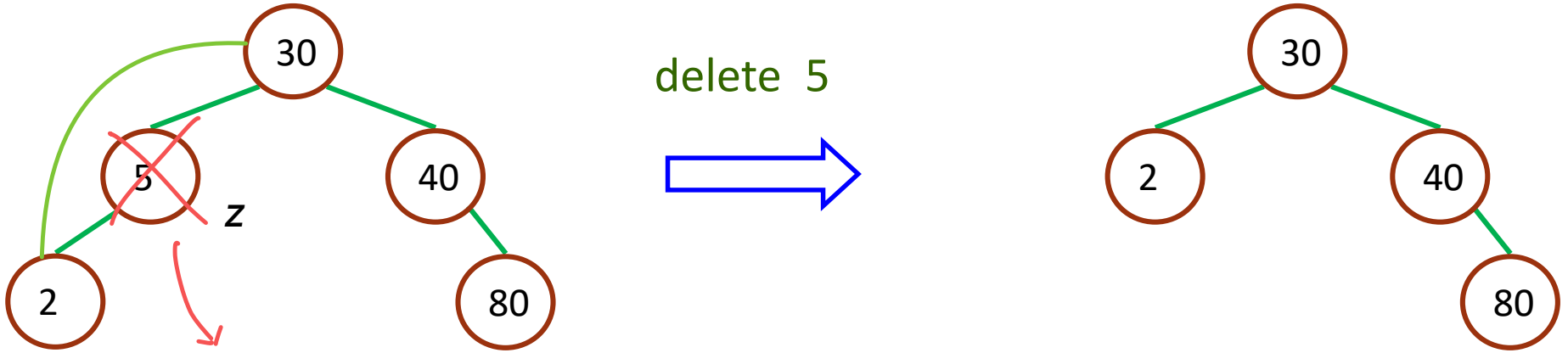
葉子，直接刪

Three cases should be considered 2/3

➢Case 2: has only one child

✓delete the node and change the pointer from the parent node to the single-child node

Case 2:



只有一个小孩，
小孩直接上位

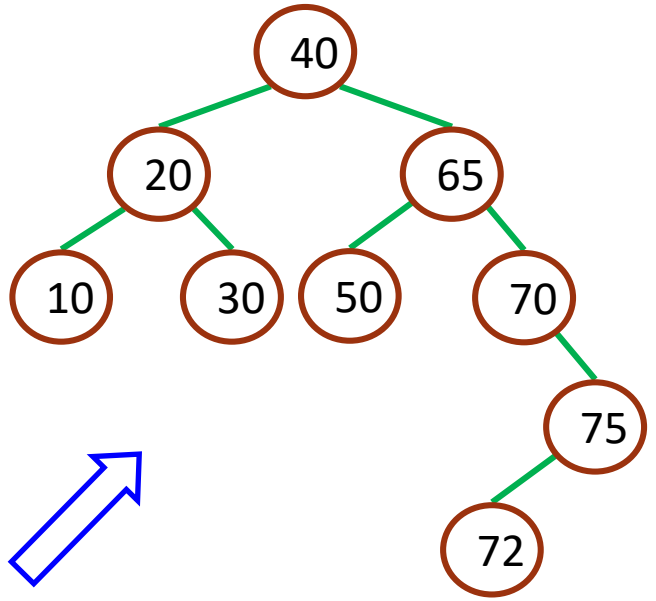
Three cases should be considered 3/3

➢Case 3: has two children

有2个小孩時，情況分成……

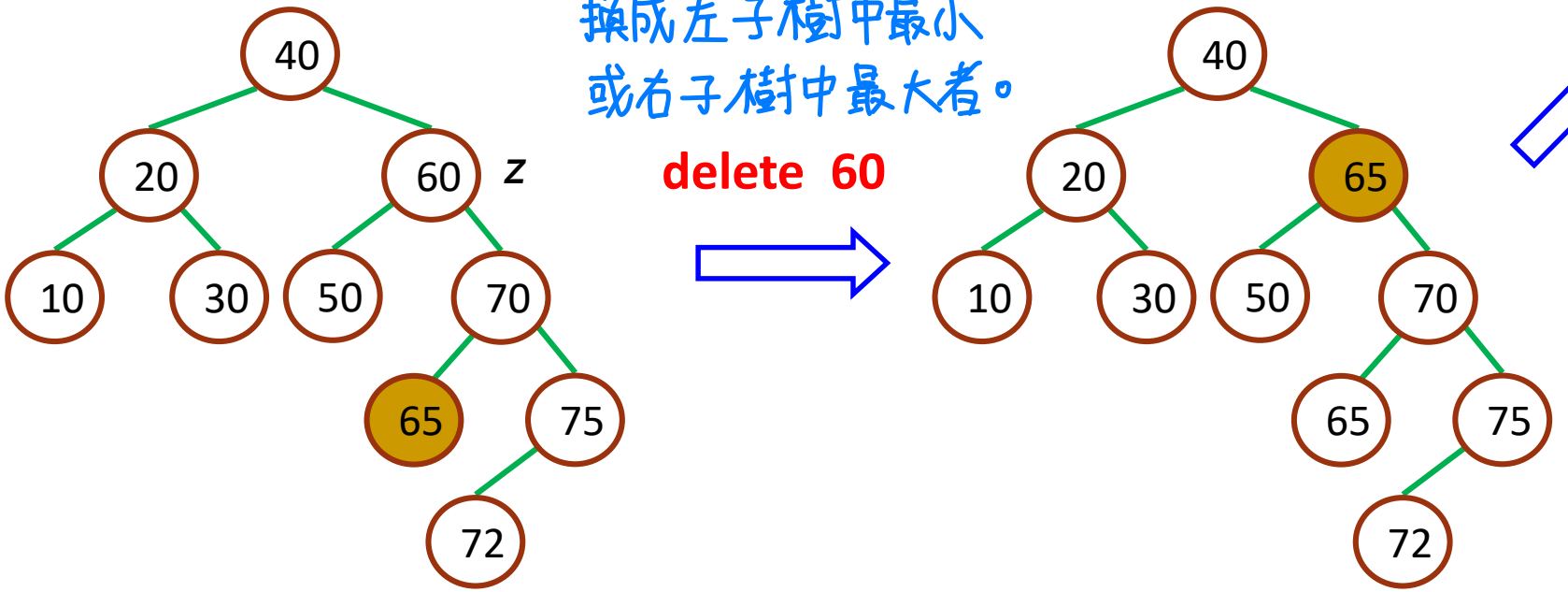
✓replaced by the largest element in its left subtree, or

✓replaced by the smallest element in its right subtree



換成左子樹中最小
或右子樹中最大者。

Case 3:

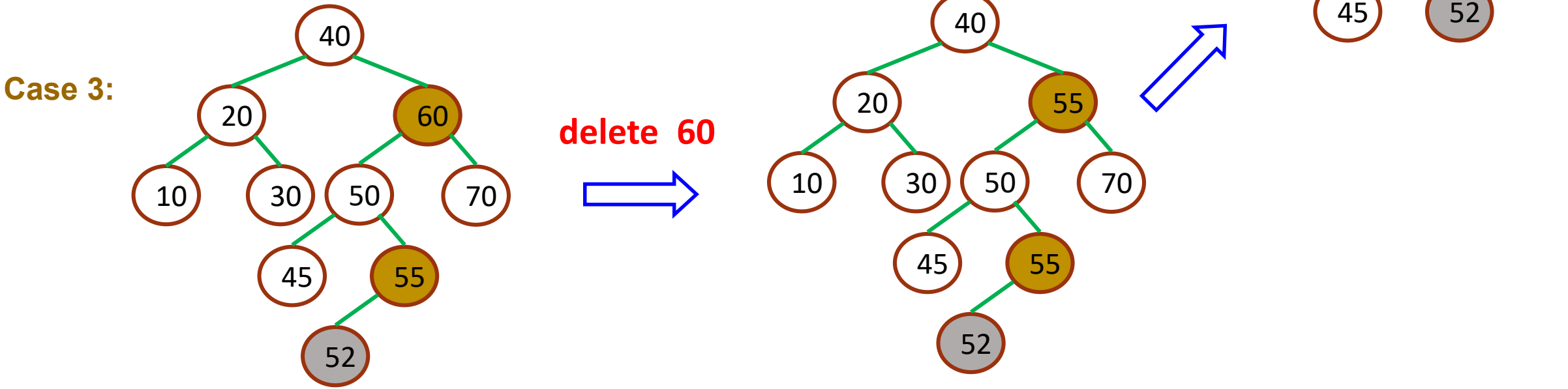


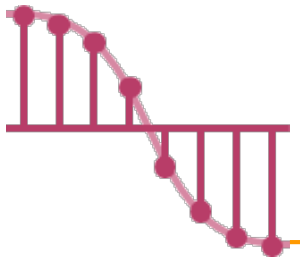
Three cases should be considered 3/3

➢Case 3: has two children

✓replaced by the largest element in its left subtree, or

✓replaced by the smallest element in its right subtree

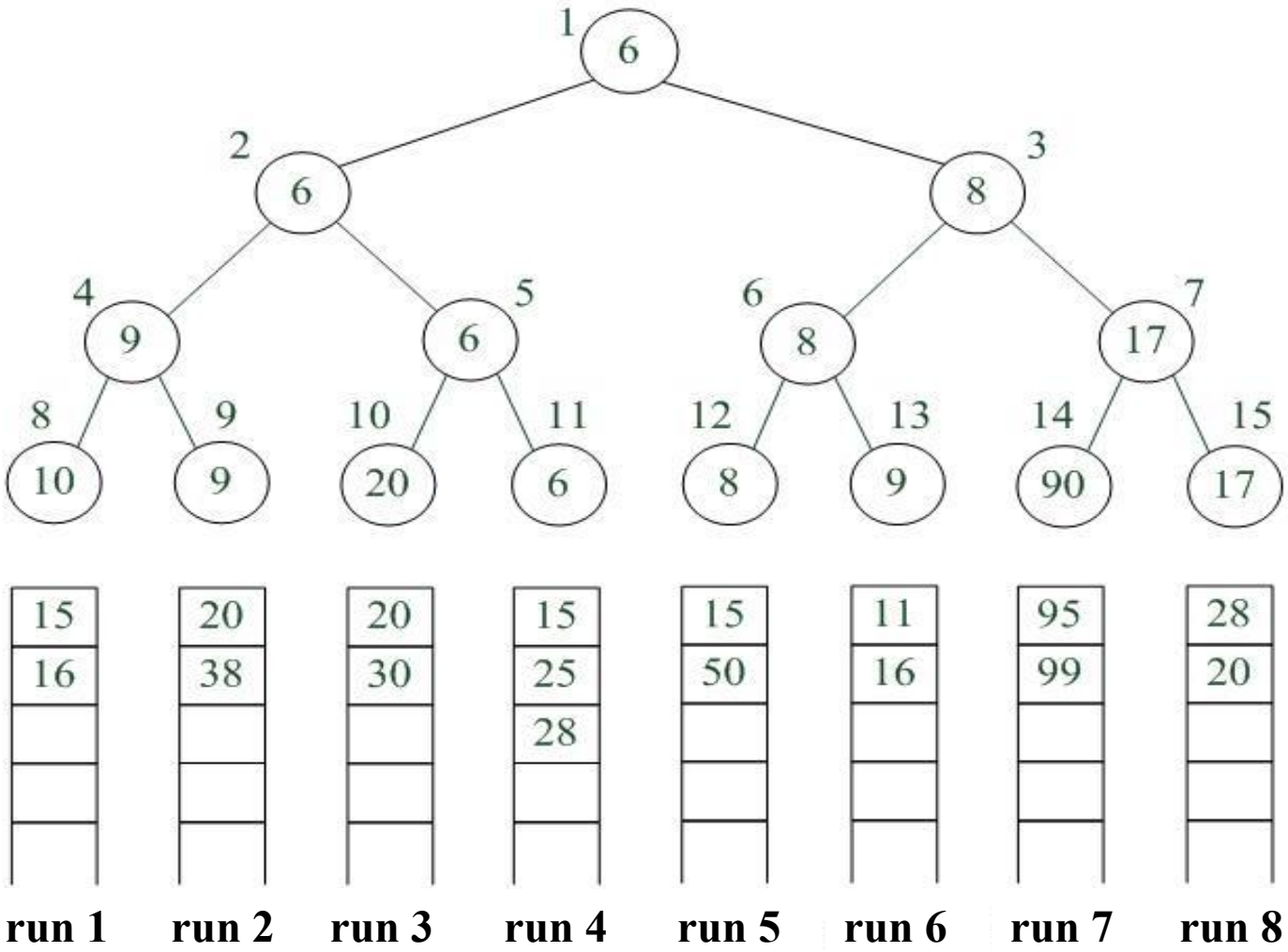




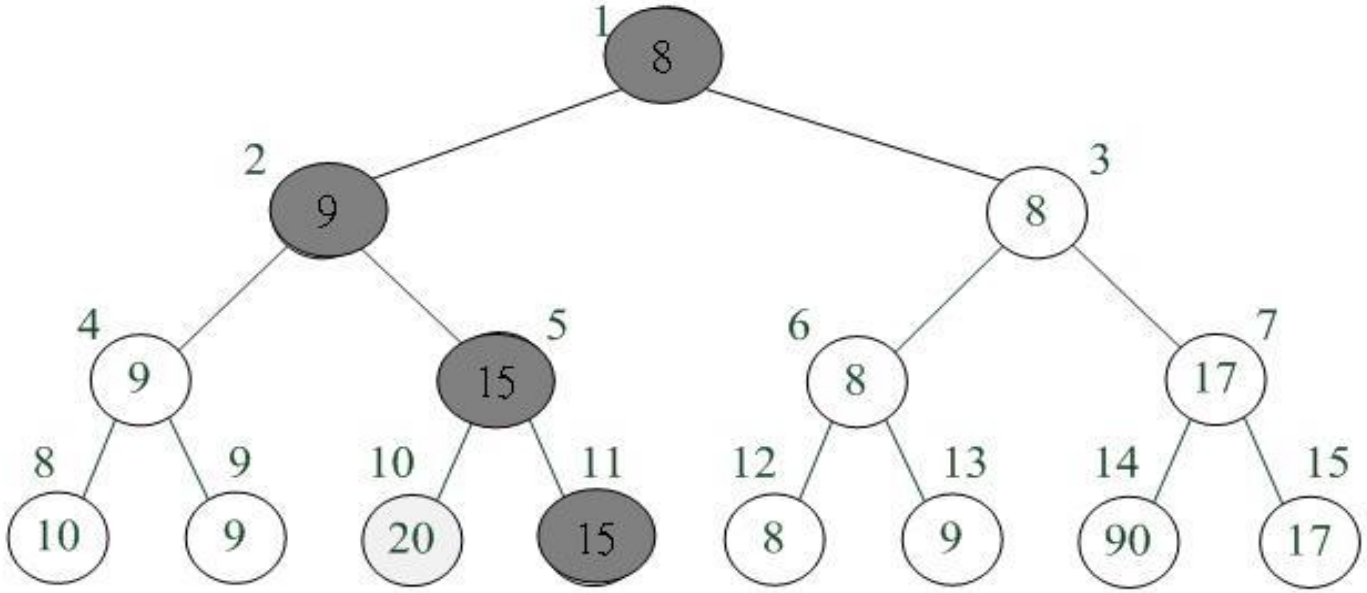
Deleting a nonleaf node that has two children

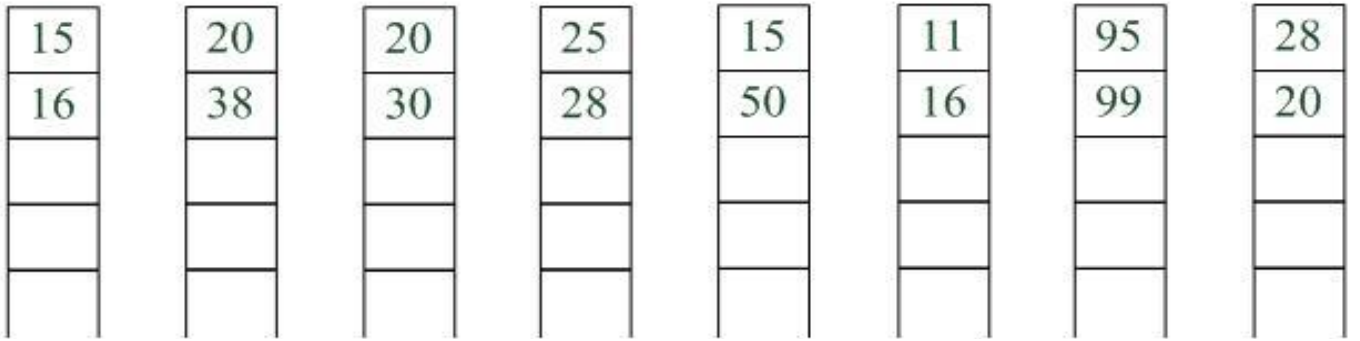
- One may verify, in both cases, it is originally in a node with a degree of at most one.
- The time complexity for case 3 is $O(h)$, where h is the height of the binary search tree.
- **A deletion can be performed in $O(h)$.**

Winner tree for $k = 8$



Winner tree for $k = 8$ (next turn)

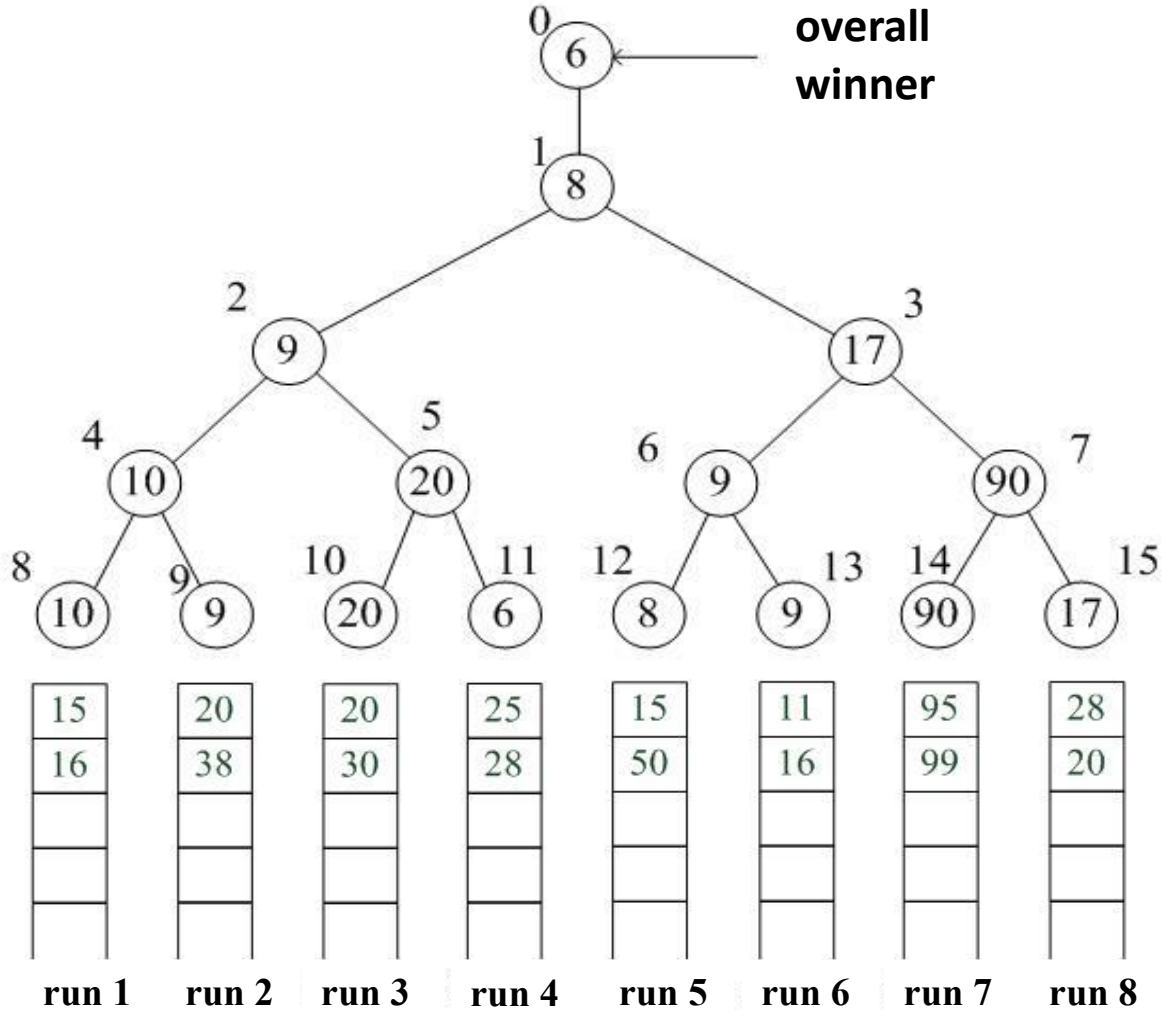




run 1 run 2 run 3 run 4 run 5 run 6 run 7 run 8

Loser tree for $k = 8$

- 敗者樹簡化了重新建構樹的過程
  - ✓ 敗者樹的重新建構只與該節點的父節點的記錄有關
  - ✓ 而勝者樹的重新建構與該節點的兄弟節點有關



Forests

➢A **forest** is a set of $n \geq 0$ disjoint trees.

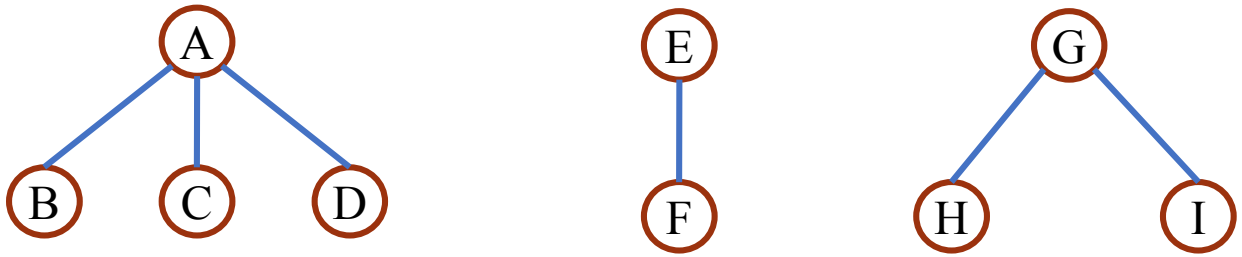
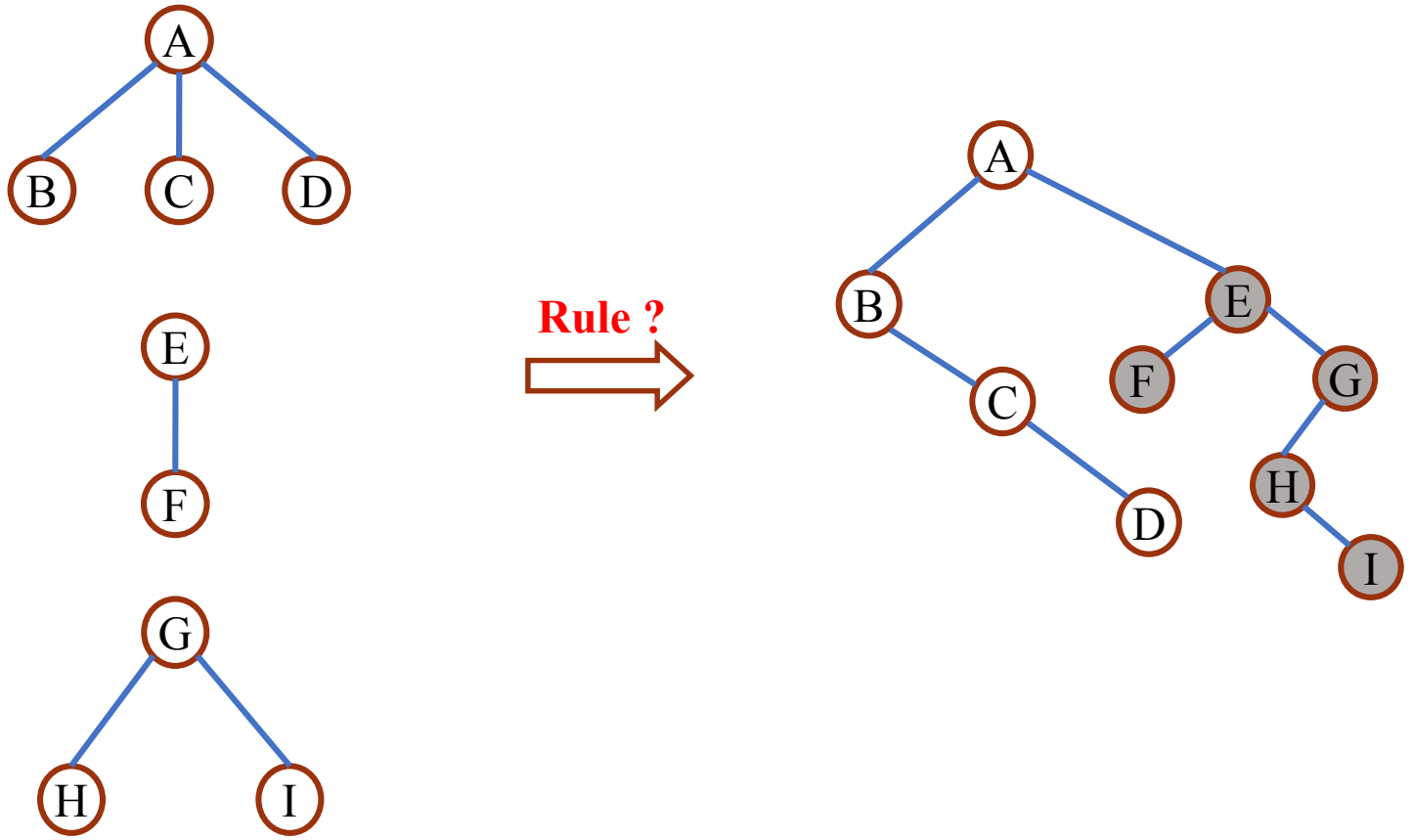


Figure 5.35：Three-tree forest

注意：forest不用相連

Binary tree representation of forest



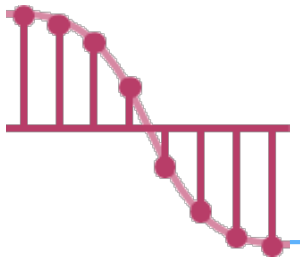
Transforming a Forest into a Binary Tree

- If $T_1,\ldots, T_n$ is a forest of trees, then the binary tree corresponding to this forest, denoted by $B(T_1,\ldots, T_n)$,
  - ✓is empty if $n = 0$
  - ✓has root equal to root(T_1);
  - ✓has left subtree equal to $B(T_{11}, T_{12}, \ldots, T_{1m})$, where $T_{11}, \ldots, T_{1m}$ are the subtrees of root (T_1); and
  - ✓has right subtree $B(T_2,\ldots, T_n)$.

！左孩子，右兄弟！

左→接原本樹中「第一个孩子」

右→接原本樹中的下一个兄弟

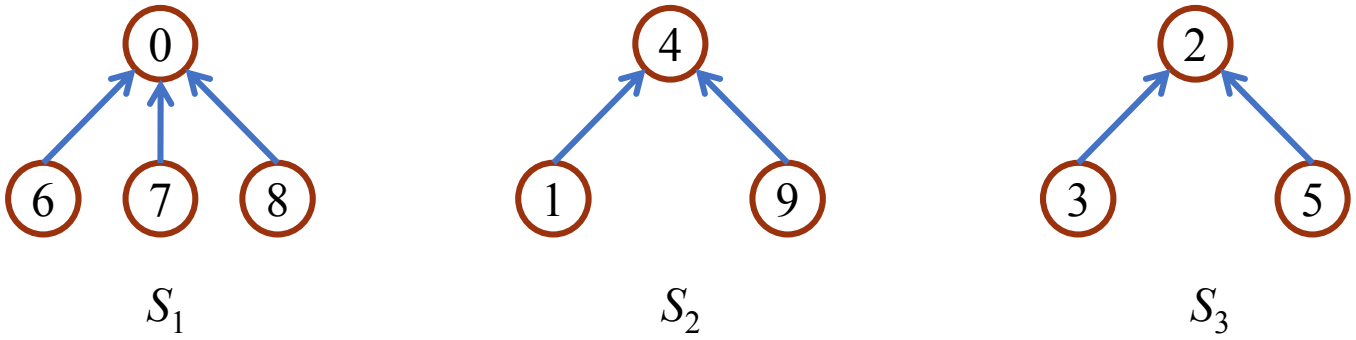


Introduction

- In this section, we study the use of trees in the representation of **sets**
- For simplicity, we assume that the elements of the sets are the numbers 0, 1, 2, … , $n-1$
- We also assume that the sets being represented are **pairwise disjoint**
- If S_i and S_j are two **disjoint** sets, then there is no element that is in both S_i and S_j

Set Representation 1/2

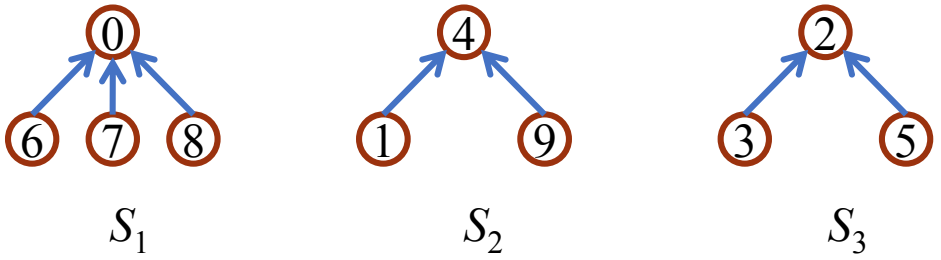
- If we have 10 elements numbered 0 through 9, we may partition them into three disjoint sets, $S_1=\{0, 6, 7, 8\}$, $S_2=\{1, 4, 9\}$, and $S_3=\{2, 3, 5\}$.
- The following figure shows one possible representation for these sets.
- Notice that for each set we have linked the nodes from the children to the parent



S_1 S_2 S_3

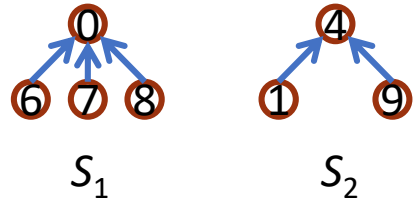
Set Representation 2/2

- The operations that we wish to perform on these sets are:
  - ✓**Disjoint set union.** If S_i and S_j are two disjoint sets, then their union $S_i \cup S_j$ = { all elements, x, such that x is in S_i or S_j }.
    - For example, $S_1 \cup S_2 = \{0, 6, 7, 8, 1, 4, 9\}$.
  - ✓**Find(i).** find the set containing the element i.
    - For example, 3 is in set S_3 and 8 is in set S_1.

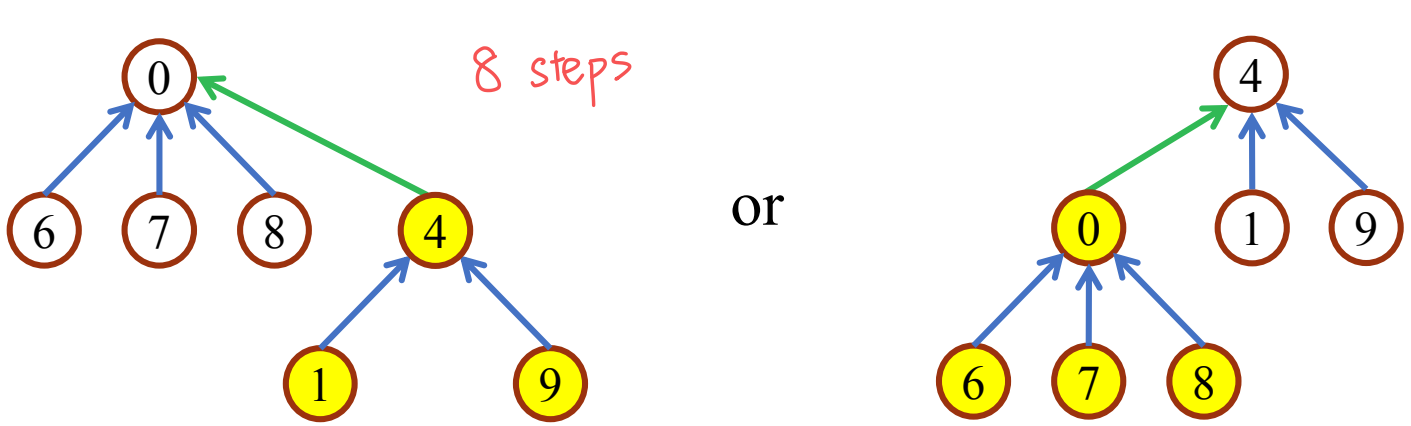


Union Operation

做聯集



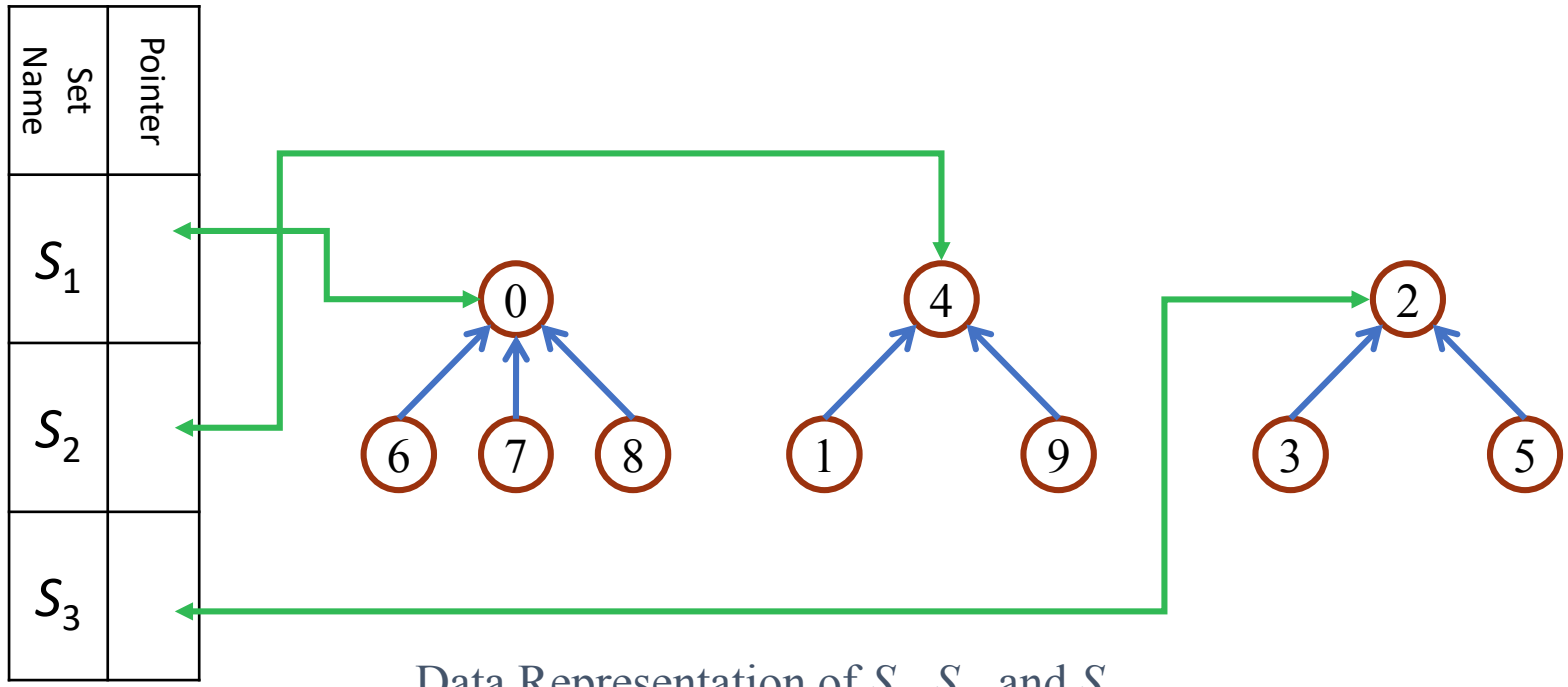
- Suppose that we wish to obtain the union of S_1 and S_2.
- Since we have linked the nodes from the children to parent, we simply make one of the trees a subtree of the other.



Possible representation of $S_1 \cup S_2$

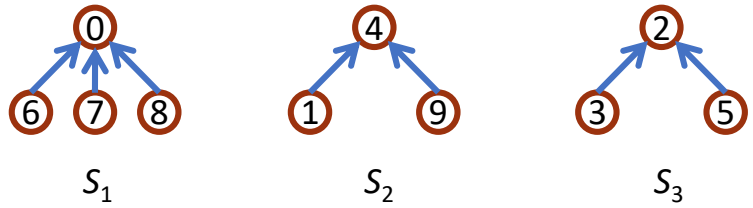
Find Operation

- We can find which set an element is in by following the parent links to the root and then returning the pointer to the set name.



Data Representation of S_1, S_2, and S_3

Array Representation of Sets



- We identify the sets by the roots of the trees representing them.
- Since the nodes in the trees are numbered 0 through $n-1$, we can use the node's number as an index.
- This means that each node needs only one field, **the index of its parent**, to link to its parent.
- The following figure shows the array representation of the sets S_1, S_2, and S_3. Notice that root nodes have a parent of -1.

	S_1	S_2	S_3	S_3	S_2	S_3	S_1	S_1	S_1	S_2
i	**[0]**	**[1]**	**[2]**	**[3]**	**[4]**	**[5]**	**[6]**	**[7]**	**[8]**	**[9]**
parent	**-1**	4	**-1**	2	**-1**	2	0	0	0	4

Union and Find Operations

- We can now **find element *i*** by simply following the parent values starting at *i* and continuing until we reach a negative parent value.
- For example, to find 5, we start at 5, and then move to 5's parent, 2. Since node 2 has a negative parent value we have reached the root.

i	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
parent	-1	4	-1	2	-1	2	0	0	0	4

- To **union** the two trees with roots *i* and *j*, we simply set parent[*i*] = *j*.

Program 5.19
Initial attempt at union-find functions

- int simpleFind (int i)

```
{
    for (; parent[i] ≥ 0; i = parent[i])
        ;
    return i;
}
```

i	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
parent	-1	4	-1	2	-1	2	0	0	0	4

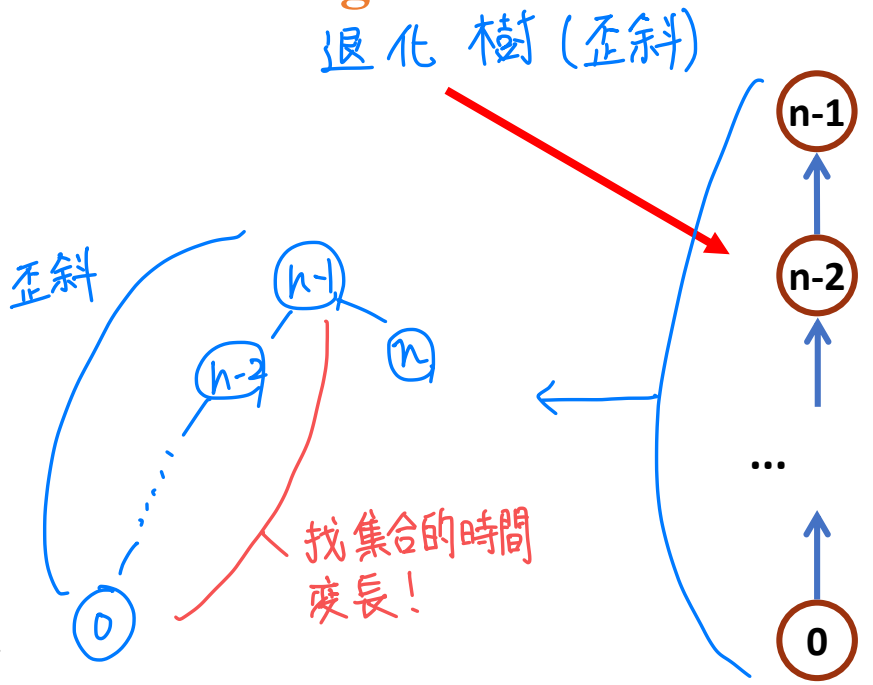
- void simpleUnion (int i, int j)

```
{
    parent [i] = j;
}
```

Analysis of Program 5.19

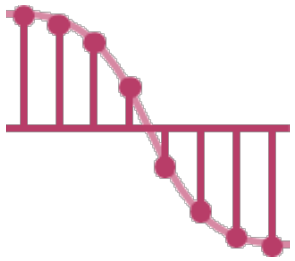
- Consider the case:
  - ✓ we start with p elements, each in a set of its own, that is $S_i = \{i\}$.
- The following sequence of union-find operations produces the **degenerate tree.**

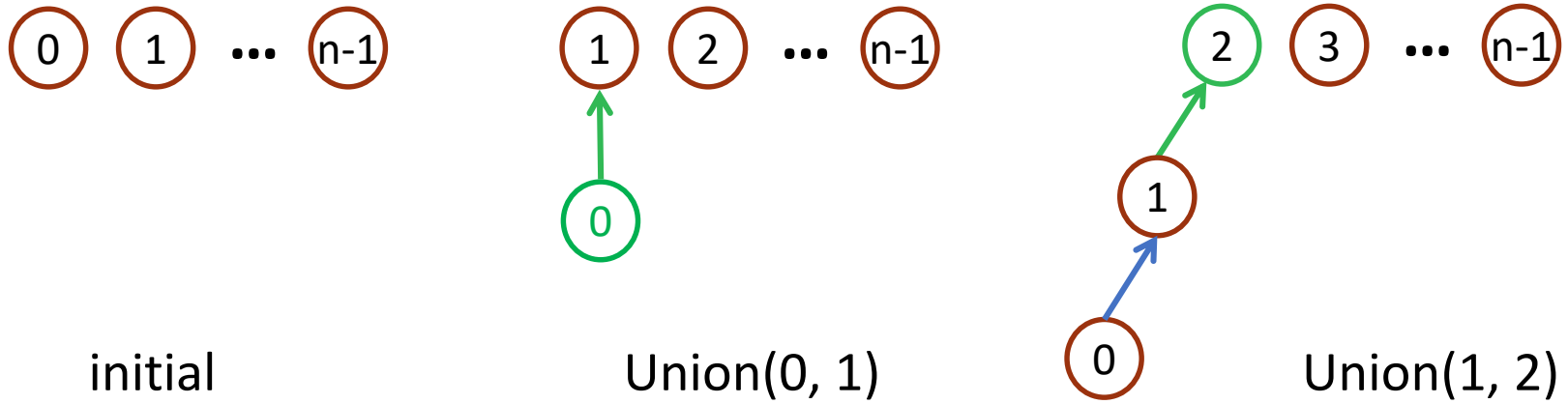
```
union(0,1), find(0)
union(1,2), find(0)
...
union(n-2, n-1), find(0)
```



- The time complexity of **union operation** is $O(n)$.

The time complexity of **find operation** is $\sum_{i=2}^{n} i = O(n^2)$.

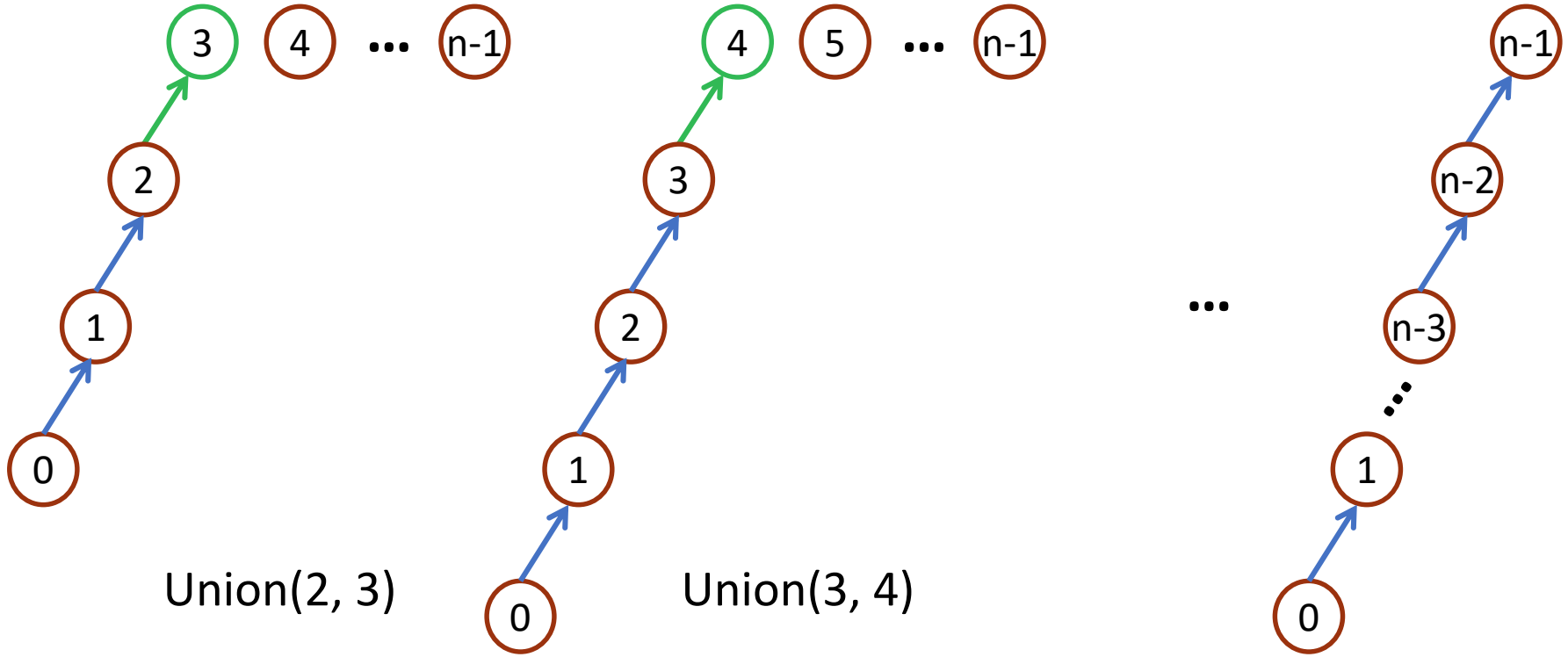




initial

Union(0, 1)

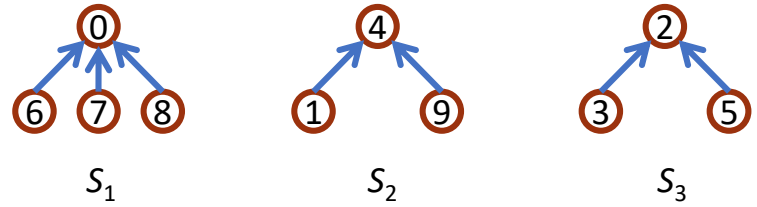
Union(1, 2)



Union(2, 3)

Union(3, 4)

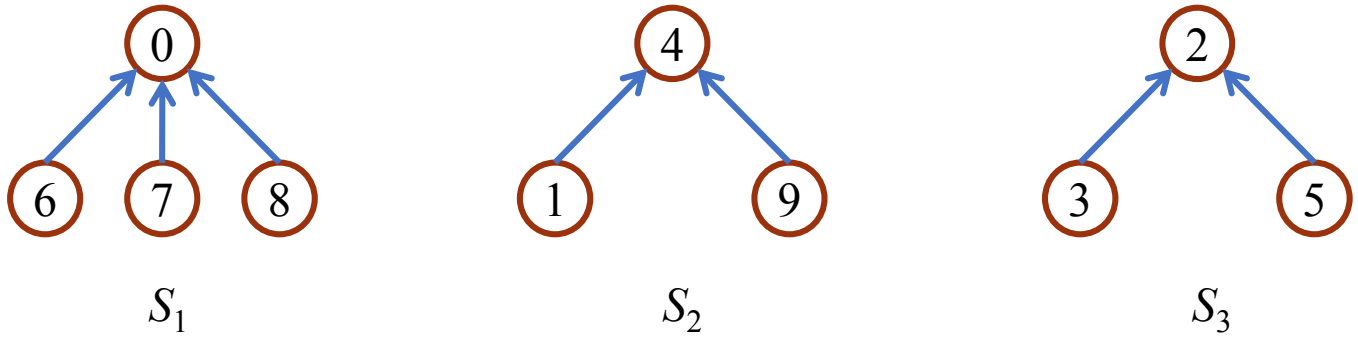
Weighting Rule for Union



- To avoid the creation of degenerate trees, we adopt the weighting rule for union(i, j).
- **Weighting rule for union(i, j).** If the number of nodes in tree i is less than the number in tree j then make j the parent of i; otherwise make i the parent of j.
- To implement the weighting rule, we set parent[i] equals the negative number of nodes in that tree, if i is a root node.

i	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
parent	-4	4	-3	2	-3	2	0	0	0	4

Example



Root nodes have a parent of -1

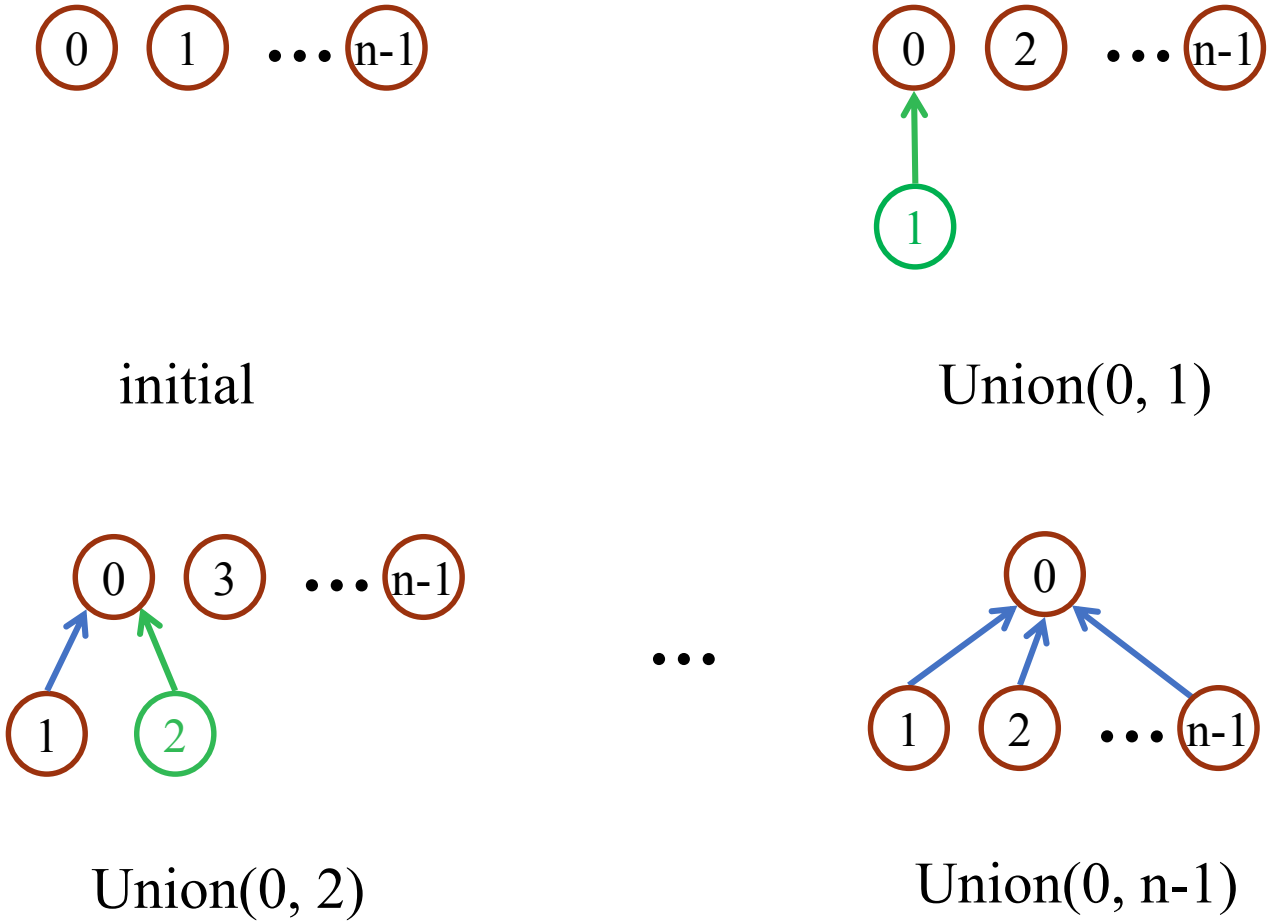
i	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
parent	-1	4	-1	2	-1	2	0	0	0	4

Parent[*i*] equals the negative number of nodes in that tree, if *i* is a root node.

i	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
parent	-4	4	-3	2	-3	2	0	0	0	4

改成記錄集合的元素

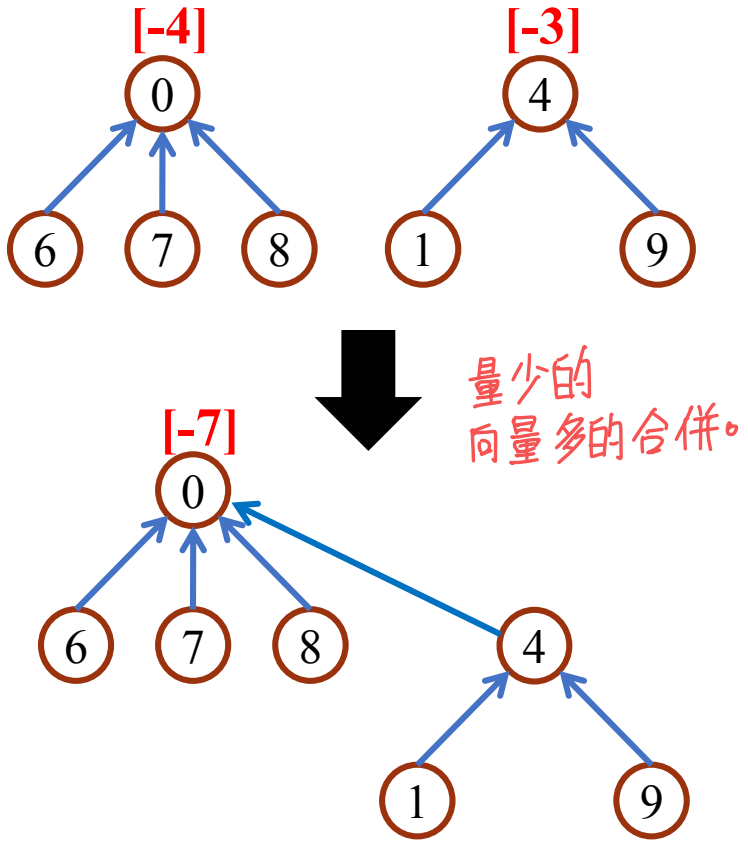
Figure 5.42 Trees obtained using the weighted rule



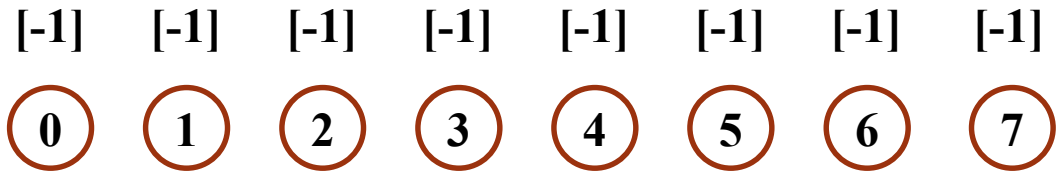
Program 5.20.
Union function using weighting rule

- void weightedUnion(int i, int j)

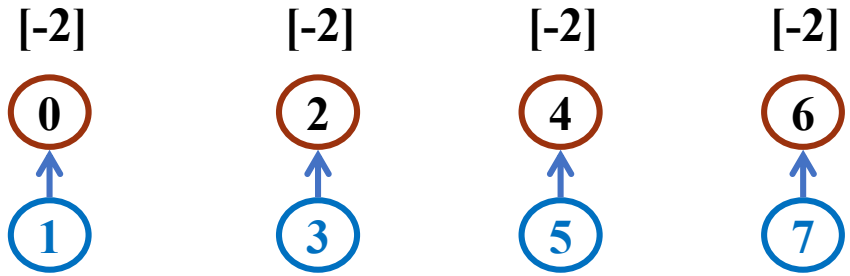
```
{
    /* union the sets with roots i and j, i != j, using the
    weighting rule. parent [i] = -count [i] and
    parent [j] = -count[j] */
    int temp = parent[i] + parent[j];
    if (parent[i] > parent[j]) {
        parent[i] = j; /*make j the new root */
        parent[j] = temp;
    }
    else {
        parent[j] = i; /*make i the new root */
        parent[i] = temp;
    }
}
```



Trees achieving worst case bound 1/2

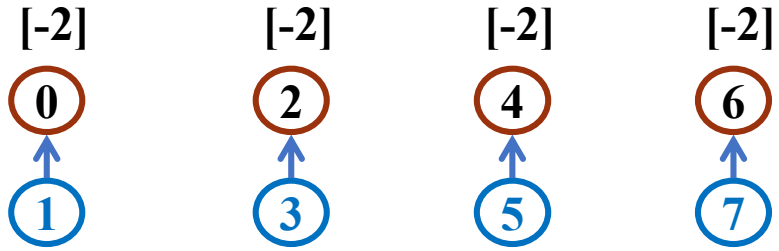


(a) Initial height-1 trees

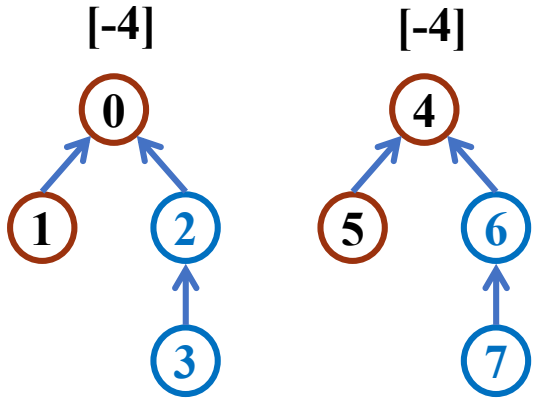


(b)Height-2 trees following Union(0, 1), (2, 3), (4, 5) and (6, 7)

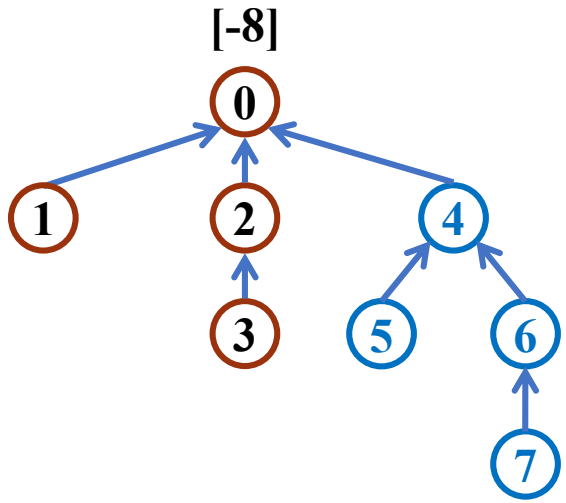
Trees achieving worst case bound 2/2



(b)Height-2 trees following Union(0, 1), (2, 3), (4, 5) and (6, 7)



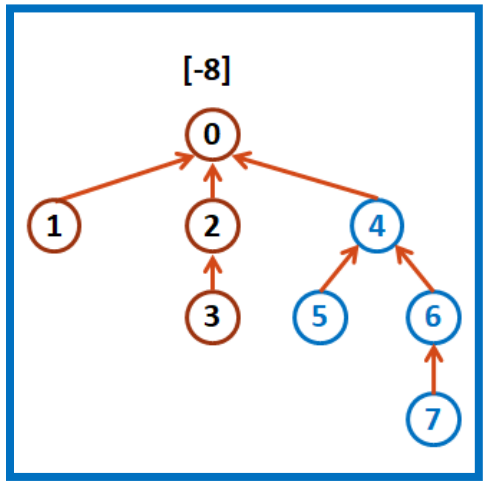
(c)Height-3 trees following Union(0, 2) and (4, 6)

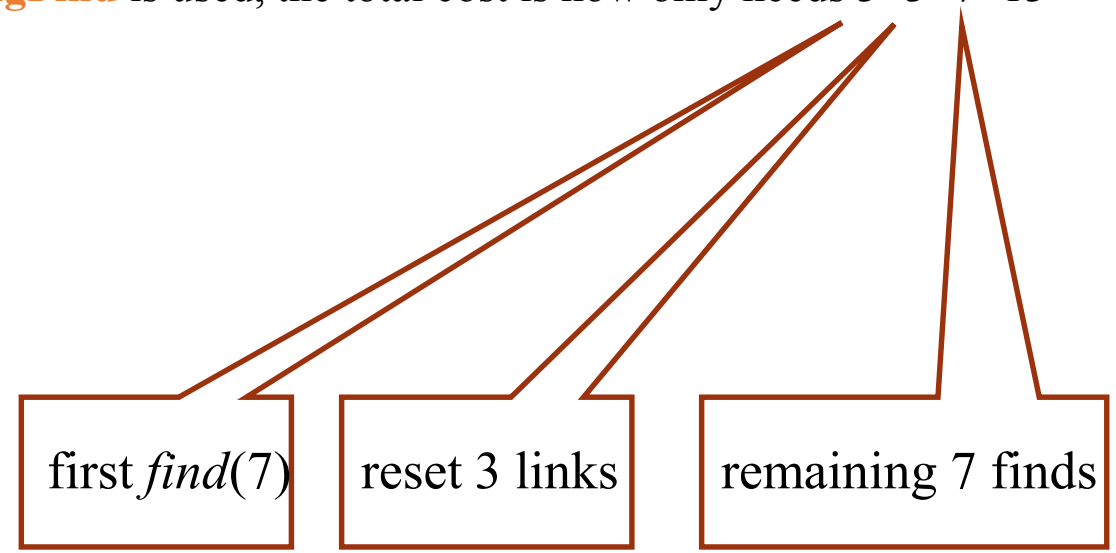


(d)Height-4 tree following Union(0, 4)

Collapsing Rule 1/2

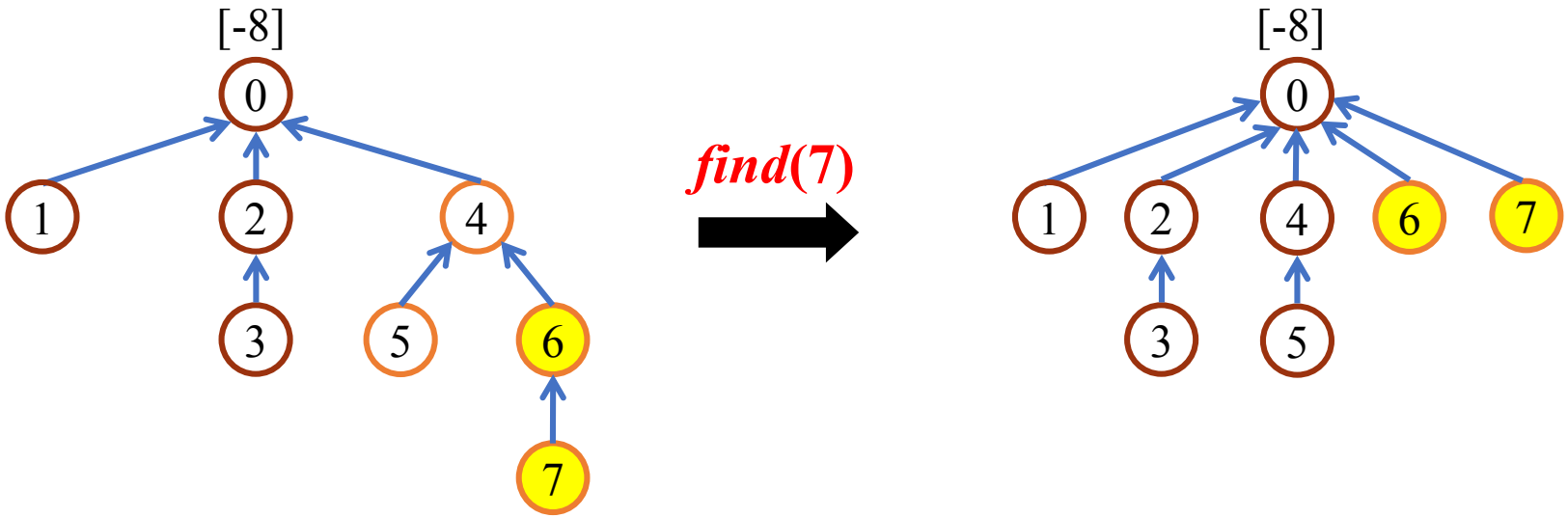
- **Collapsing rule (瓦解規則):** If j is a node on the path from i to its root and parent[i] ≠ root(i), then set parent[j] to root(i).
- Consider the previous example.
  - ✓ Now process the following eight finds: *find*(7), *find*(7), …, *find*(7).
    The **simpleFind** needs 3*8 = 24 moves. When **collapsingFind** is used, the total cost is now only needs 3+3+7=13 moves.





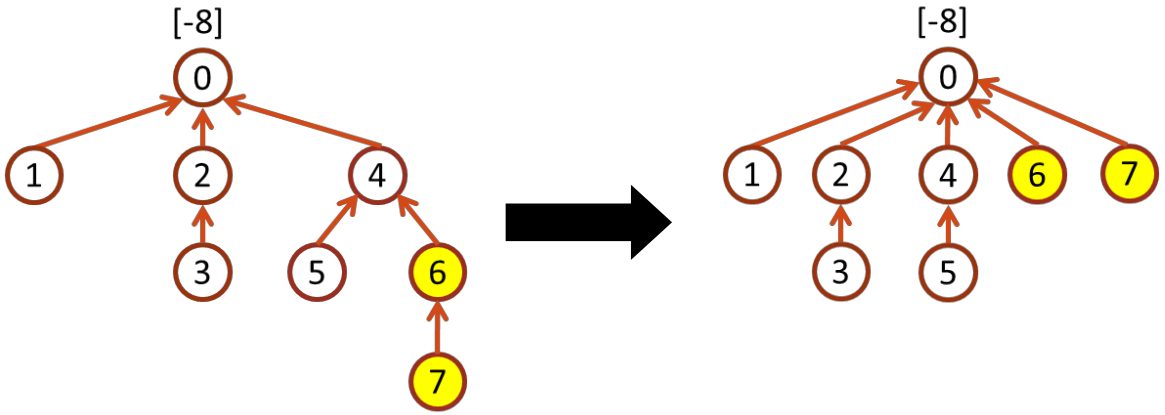
Collapsing Rule 2/2

- When collapsingFind is used, the first *find*(7) requires going up three links and then resetting two links.
- Note that even though only two parent links need to be reset, function collapsingFind will actually reset three (the parent of 4 is reset to 0).



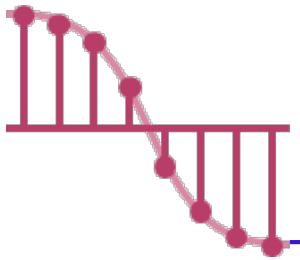
Program 5.21 Collapsing rule

```
int collapsingFind (int i)
{
    /* find the root of the tree containing element i.
       Use the collapsing rule to collapse all nodes
       from i to root */
    int root, trail, lead;
    for (root =i; parent[root] >= 0; root  = parent[root])
        ;
    for (trail = i; trail != root; trail = lead) {
        lead = parent[trail];
        parent[trail] = root;
    }
    return root;
}
```




```
i = 7
 for (root =i; parent[root] >= 0;
root  = parent[root])
;
root = 0

trail = 7
lead = parent[7] = 6
parent[trail] = parent[7] = 0
trail = 6
lead = parent[6] = 4
parent[6] = 0
trail = 4
lead = parent[4] = 0
parent[4] = 0
trail = 0
```



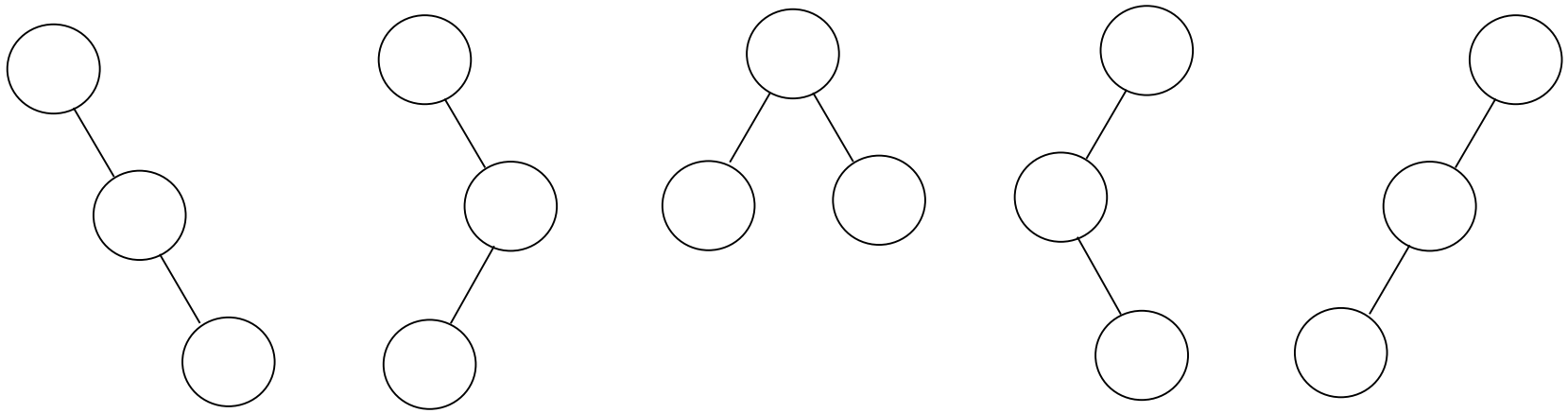
Counting Binary Tree

- We consider the following three disparate problems:
  - ✓The number of distinct binary trees having n nodes
  - ✓The number of distinct permutations of the numbers from 1 through n obtainable by a stack
  - ✓The number of distinct ways of multiplying n+1 matrices
- **They amazingly have the same solution.**

Counting Binary Tree

- We consider the following three disparate problems:
  - ✓The number of distinct binary trees having *n* nodes

Example. If *n*=3,



There are 5 distinct binary tree

Counting Binary Tree

- We consider the following three disparate problems:

不考

  - ✓The number of distinct permutations of the numbers from 1 through *n* obtainable by a stack

Example. If *n*=3,

push 1 ➔ pop ➔ push 2 ➔ pop ➔ push 3 ➔ pop	**123**
push 1 ➔ pop ➔ push 2 ➔ push 3 ➔ pop ➔ pop	**132**
push 1 ➔ push 2 ➔ push 3 ➔ pop ➔ pop ➔ pop	**321**
push 1 ➔ push 2 ➔ pop ➔ pop ➔ push 3 ➔ pop	**213**
push 1 ➔ push 2 ➔ pop ➔ push 3 ➔ pop ➔ pop	**231**

Counting Binary Tree

- We consider the following three disparate problems:

  - ✓The number of distinct ways of multiplying n+1 matrices

Example. If n=3,

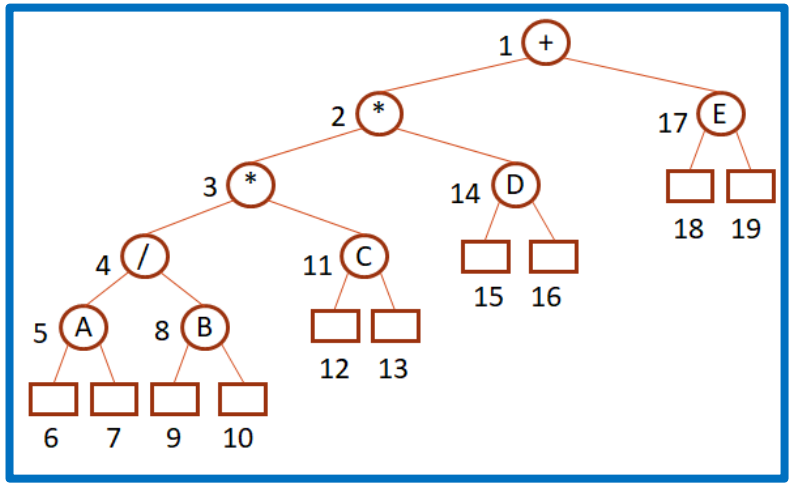
$$((M_1 * M_2) * M_3) * M_4$$
$$(M_1 * (M_2 * M_3)) * M_4$$
$$M_1 * ((M_2 * M_3) * M_4)$$
$$M_1 * (M_2 * (M_3 * M_4))$$
$$(M_1 * M_2) * (M_3 * M_4)$$

Stack permutation 1/4

会考

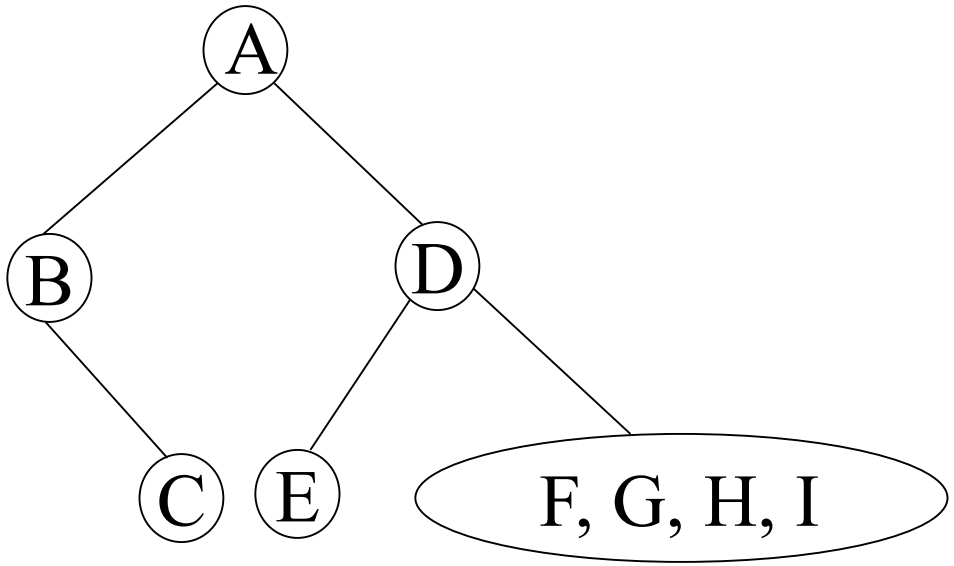
- In section 5.3 we introduced **preorder**, **inorder**, and **postorder** traversal of a binary tree and indicated that each traversal requires a stack.
- **Every binary tree has a unique pair of preorder/inorder sequences.**
- The number of **distinct binary trees is equal to** the number of **inorder permutations** obtainable from binary trees having the **preorder permutation**, 1 , 2 , … , *n*.

Inorder traversal: A/B*C*D+E (**LVR**)
Preorder traversal: +**/ABCDE (**VLR**)
Postorder traversal: AB/C*D*E+ (**LRV**)

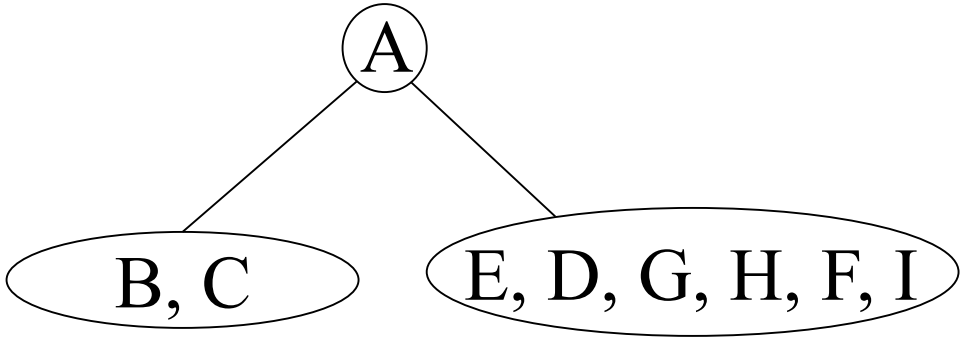


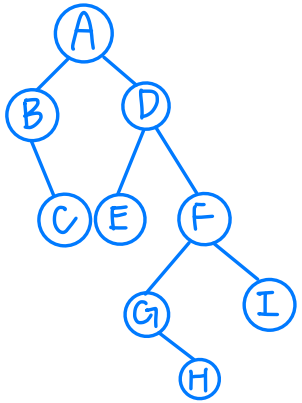
Stack permutation 2/4

preorder: A B C (D E F G H I)
inorder: B C A (E D G H F I)



preorder: A B C D E F G H I
inorder: B C A E D G H F I





Stack permutation 3/4

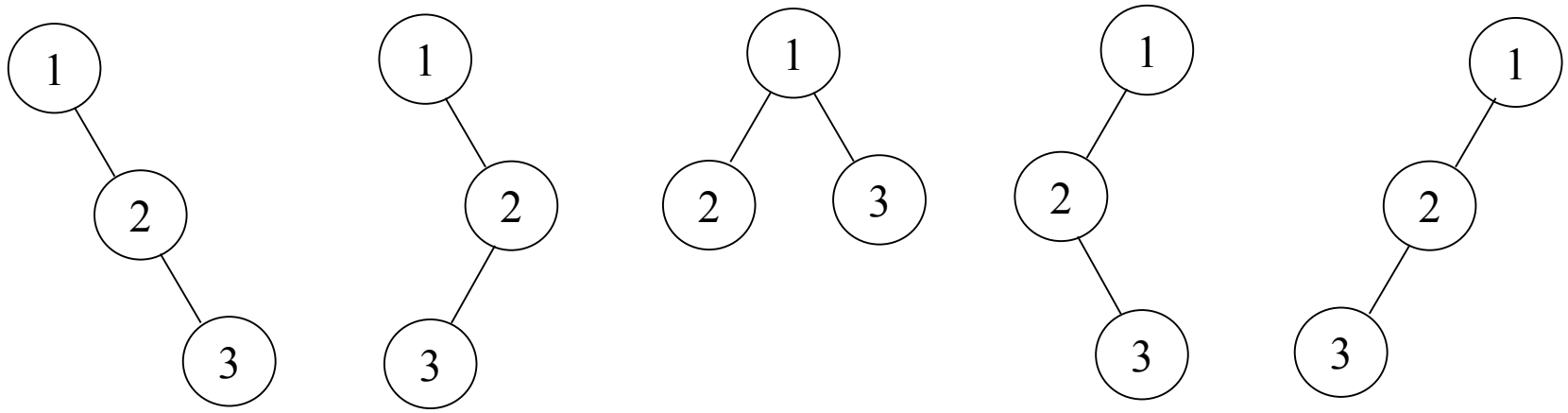
- So, we can show that the number of distinct permutations obtainable by passing the numbers 1 through *n* through a stack is equal to the number of distinct binary trees with *n* nodes.
- Example : if we start with the numbers 1, 2, and 3, then the possible permutations obtainable by a stack are

(1,2,3) (1,3,2) (2,1,3) (2,3,1) (3,2,1).

Example. If *n*=3,

push 1 → pop → push 2 → pop → push 3 → pop	123
push 1 → pop → push 2 → push 3 → pop → pop	132
push 1 → push 2 → push 3 → pop → pop → pop	321
push 1 → push 2 → pop → pop → push 3 → pop	213
push 1 → push 2 → pop → push 3 → pop → pop	231

Stack permutation 4/4



(1, 2, 3) (1, 3, 2) (2, 1, 3) (2, 3, 1) (3, 2, 1)

Binary trees corresponding to five permutation.

Matrix Multiplication 1/2

- The number of distinct binary trees is equal to the number of distinct inorder permutations obtainable from binary trees having the preorder permutation, 1, 2, …, *n*.
- Computing the product of n matrices are related to the distinct binary tree problem.

$$M_1 * M_2 * \ldots * M_n$$

$n = 3$	$(M_1 * M_2) * M_3$
	$M_1 * (M_2 * M_3)$
$n = 4$	$((M_1 * M_2) * M_3) * M_4$
	$(M_1 * (M_2 * M_3)) * M_4$
	$M_1 * ((M_2 * M_3) * M_4)$
	$M_1 * (M_2 * (M_3 * M_4))$
	$(M_1 * M_2) * (M_3 * M_4)$

Matrix Multiplication 2/2

➢Let b_n be the **number of different ways** to compute the product of n matrices.

Let $b_1 = 1$, $b_2 = 1$, $b_3 = 2$, and $b_4 = 5$.

$$b_n = \sum_{i=1}^{n-1} b_i b_{n-i}, \qquad n > 1$$

$$(M_1 * M_2 * \ldots * M_i) * (M_{i+1} * \ldots * M_n)$$

Example.

$$b_3 = b_1 b_2 + b_2 b_1 = 1 + 1 = 2$$

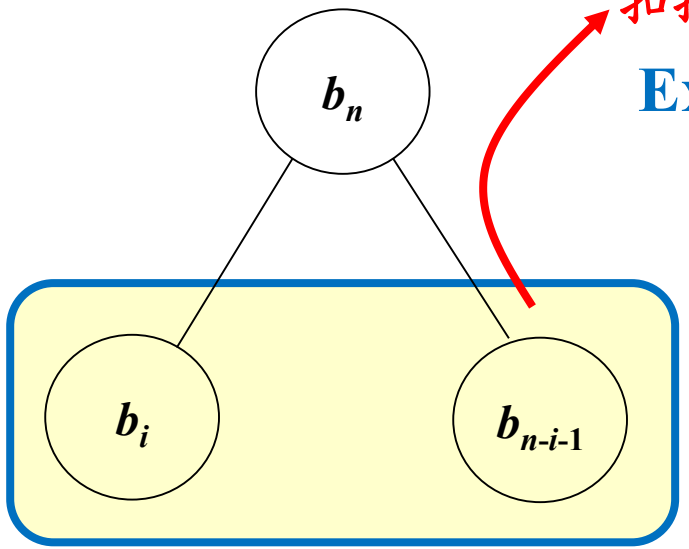
$$b_4 = b_1 b_3 + b_2 b_2 + b_3 b_1 = 2 + 1 + 2 = 5$$

$$\ldots$$

Distinct Binary Trees 1/3

➢Similarly, the number of **distinct binary trees** of n nodes is

$$b_n = \sum_{i=0}^{n-1} b_i b_{n-i-1}, \qquad n \geq 1, \text{and } b_0 = 1$$



扣掉1個做為root節點

Example.

$$b_1 = b_0 b_0 = 1$$

$$b_2 = b_0 b_1 + b_1 b_0 = 1 + 1 = 2$$

$$b_3 = b_0 b_2 + b_1 b_1 + b_2 b_0 = 2 + 1 + 2 = 5$$

...

How to compute b_n?

Distinct Binary Trees 2/3

- **Trick: Assume we let $B(x) = \sum_{i \geq 0} b_i x^i$ which is the generating function for the number of binary trees.**
- By the recurrence relation we get $$xB^2(x) = B(x) - 1$$

$$B(x) = \frac{1 - \sqrt{1-4x}}{2x}$$

二項式定理

$$(-4x)^n = (-1)^n 2^{2n} x^n$$

$$B(x) = \frac{1}{2x}\left(1 - \sum_{n \geq 0} \binom{1/2}{n} (-4x)^n\right) = \sum_{m \geq 0} \binom{1/2}{m+1} (-1)^m 2^{2m+1} x^m$$

$$b_n = \frac{1}{n+1}\binom{2n}{n} \quad \approx \quad b_n = O(4^n / n^{3/2})$$

Distinct Binary Trees 3/3

$$b_n = \binom{1/2}{n+1}(-1)^n 2^{2n+1}$$

$$= \frac{\frac{1}{2}\left(\frac{1}{2}-1\right)\left(\frac{1}{2}-2\right)\cdots\left(\frac{1}{2}-n\right)}{(n+1)(n)(n-1)(n-2)\cdots 1}(-1)^n 2^{2n+1}$$

$$= \frac{\frac{1}{2}\left(1-\frac{1}{2}\right)\left(2-\frac{1}{2}\right)\cdots\left(n-\frac{1}{2}\right)}{(n+1)(n)(n-1)(n-2)\cdots 1}2^{2n+1}$$

$$= \left(\frac{1}{2}\right)^{n+1}\frac{1\,(2-1)(4-1)\cdots(2n-1)}{(n+1)(n)(n-1)(n-2)\cdots 1}2^{2n+1}$$

$$= \frac{1\,(2-1)(4-1)\cdots(2n-1)}{(n+1)(n)(n-1)(n-2)\cdots 1}2^{n}$$

$$= \frac{1\,(1)(3)\cdots(2n-1)}{(n+1)(n)(n-1)(n-2)\cdots 1}2^{n}$$

$$= \frac{1\,(1)(3)\cdots(2n-1)}{(n+1)(n)(n-1)(n-2)\cdots 1}2^{n}\frac{2\cdot 4\cdot 6\cdot 8\cdots(2n)}{2\cdot 4\cdot 6\cdot 8\cdots(2n)}$$

$$= \frac{1\,(1)(3)\cdots(2n-1)}{(n+1)(n)(n-1)(n-2)\cdots 1}2^{n}\frac{2\cdot 4\cdot 6\cdot 8\cdots(2n)}{2^n\cdot 1\cdot 2\cdot 3\cdots(n)}$$

$$= \frac{(2n)!}{(n+1)(n!)}2^n\frac{1}{2^n\cdot(n!)} = \frac{1}{n+1}\binom{2n}{n}$$

Example.

$$b_1 = \frac{1}{2}\binom{2}{1} = \frac{2}{2} = 1$$

$$b_2 = \frac{1}{3}\binom{4}{2} = \frac{6}{3} = 2$$

$$b_3 = \frac{1}{4}\binom{6}{3} = \frac{20}{4} = 5$$

...

$$\approx \quad b_n = O(4^n / n^{3/2})$$